

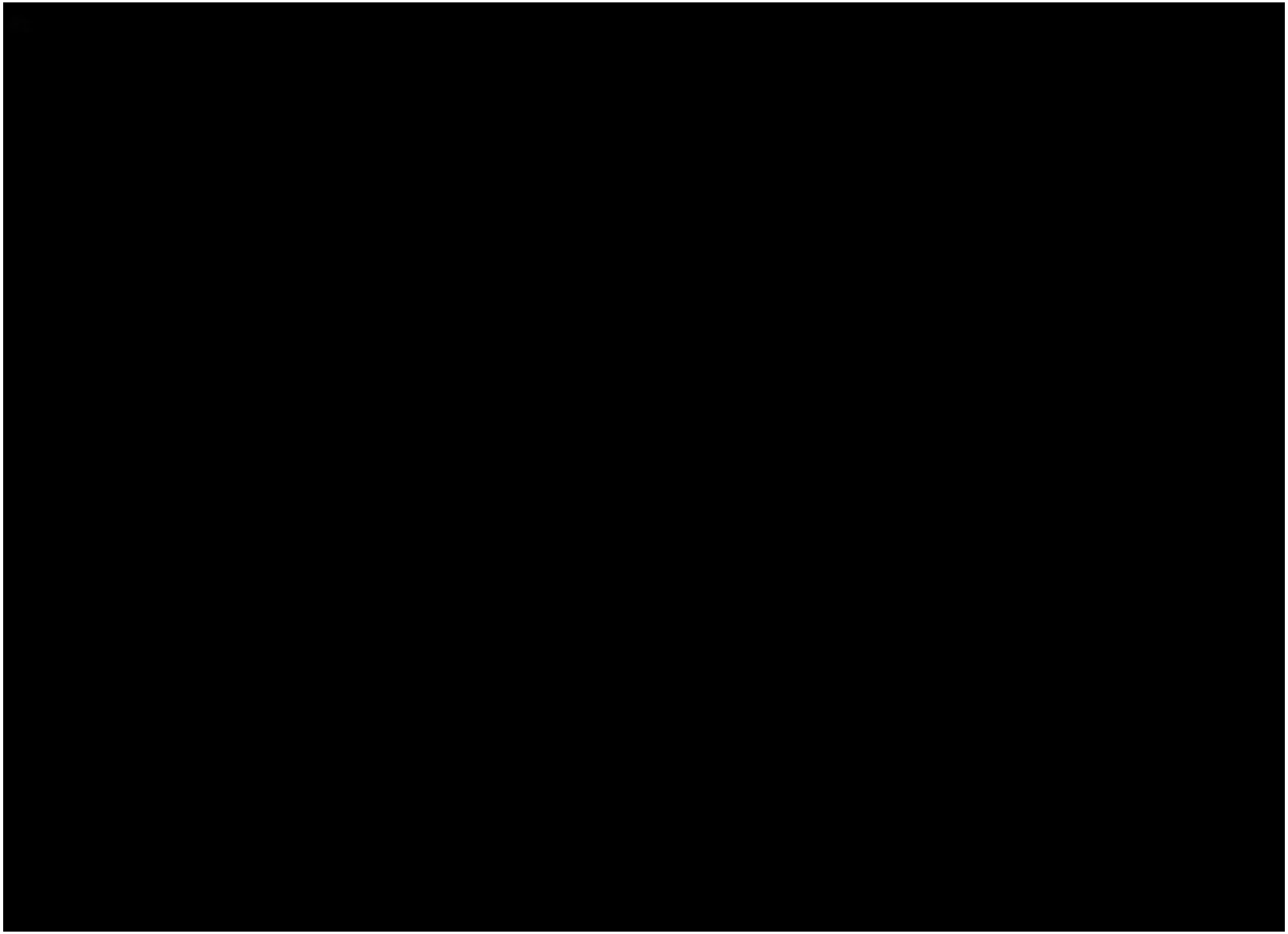
Тверской
государственный
технический
университет

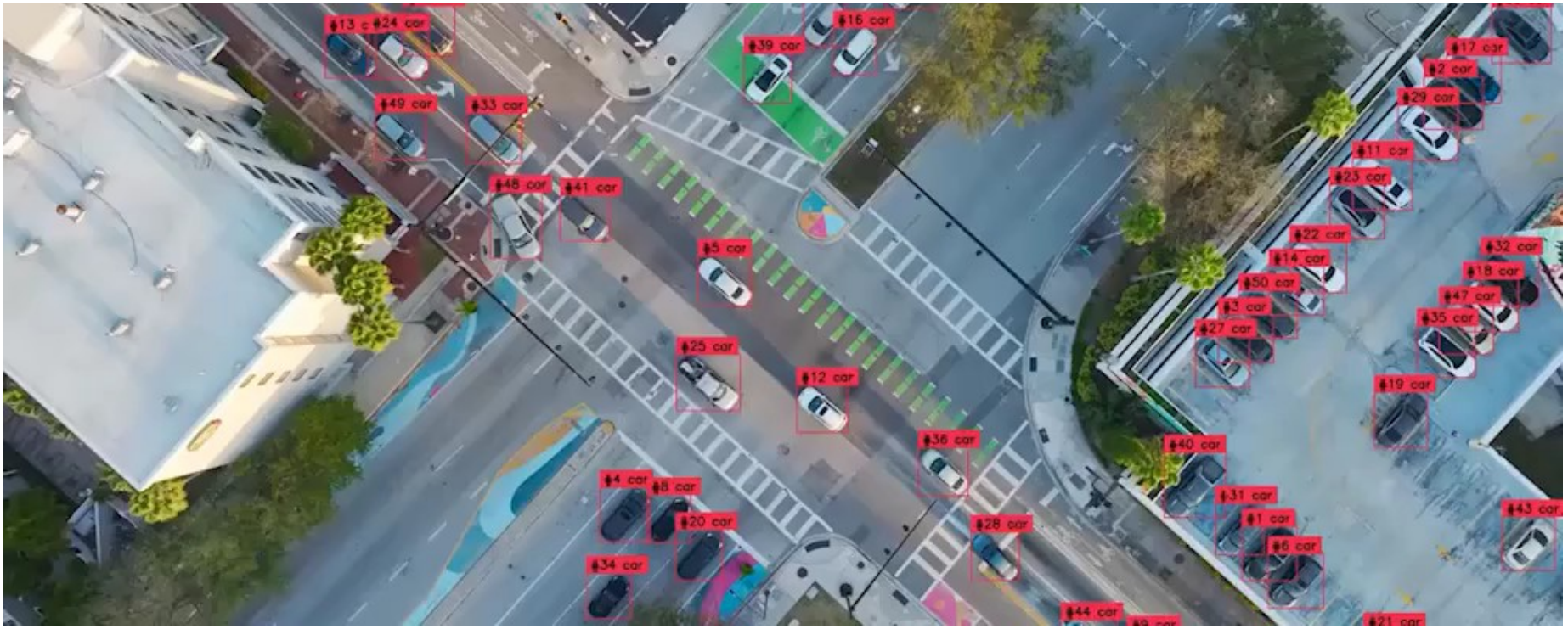
Интеллектуальные информационные системы

Сверточные нейронные сети

2026 г.

В 1989 году Ян ЛеКун представил революционную сверточную нейросеть, способную распознавать цифры в различных стилях написания. Это было логическим продолжением его идеи, предложенной годом ранее, в 1988-м.

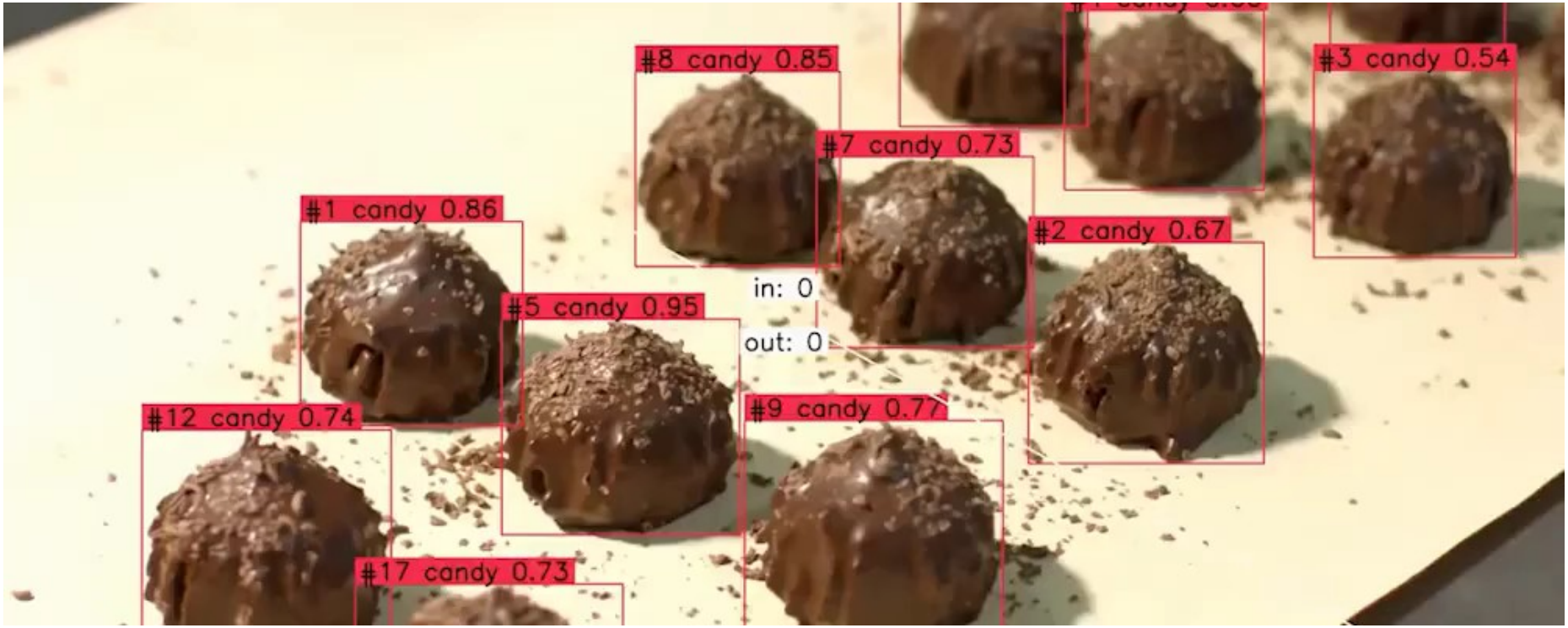




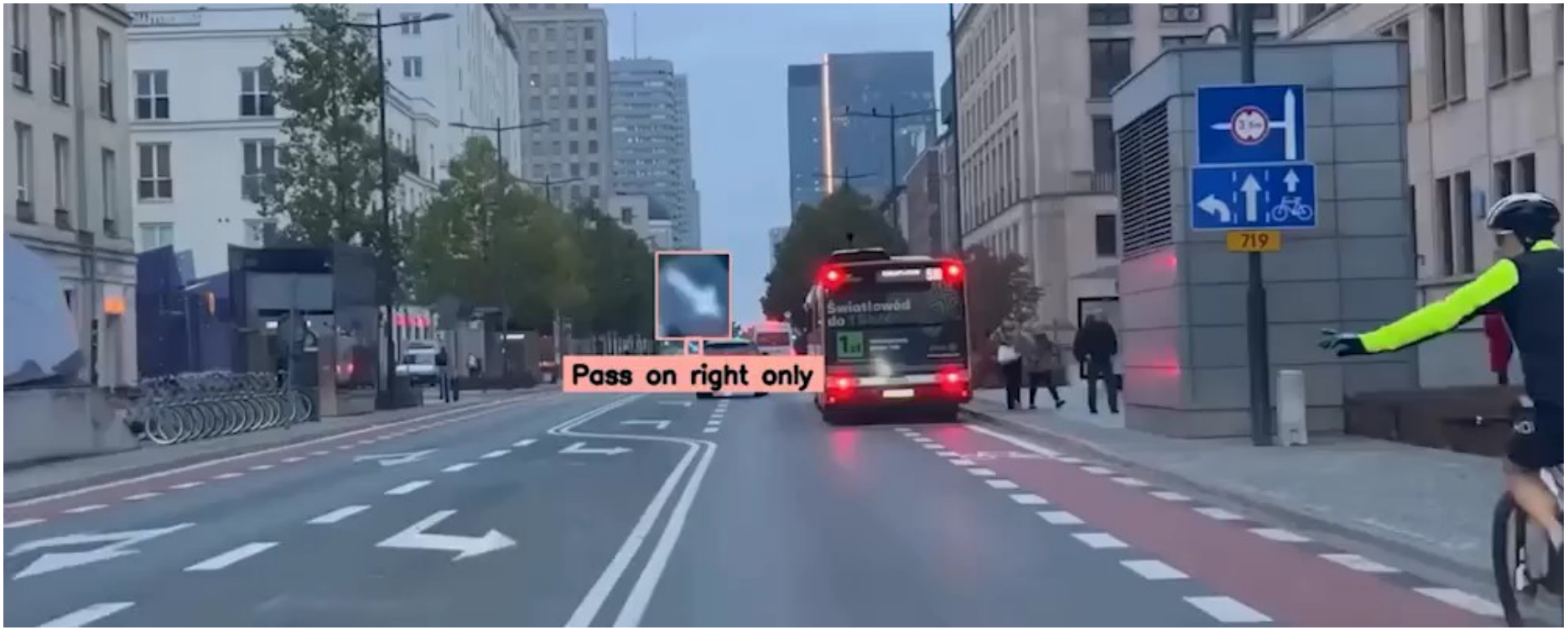
Источник: https://roboflow.com/



Источник: https://roboflow.com/

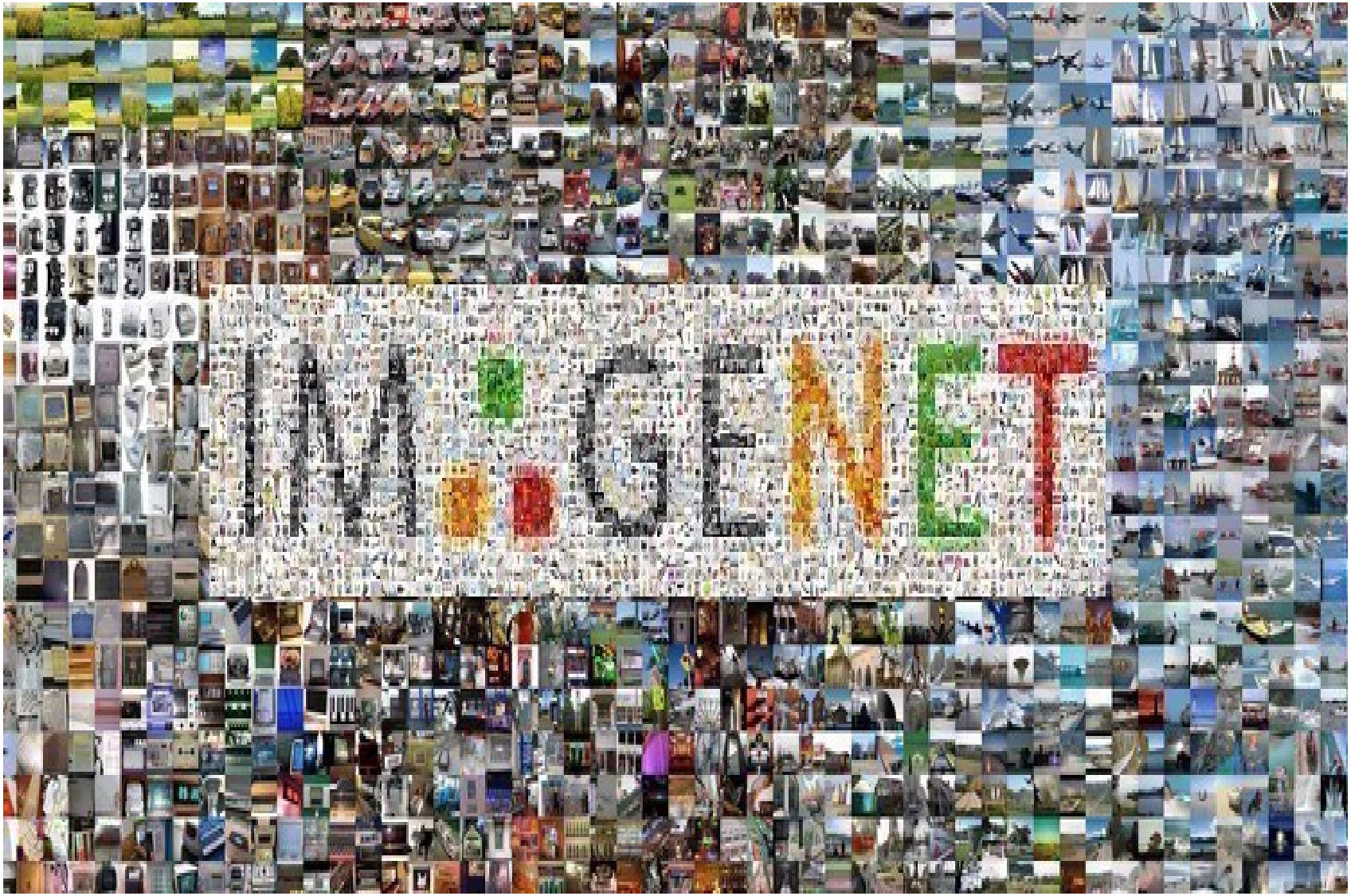


Источник: https://roboflow.com/

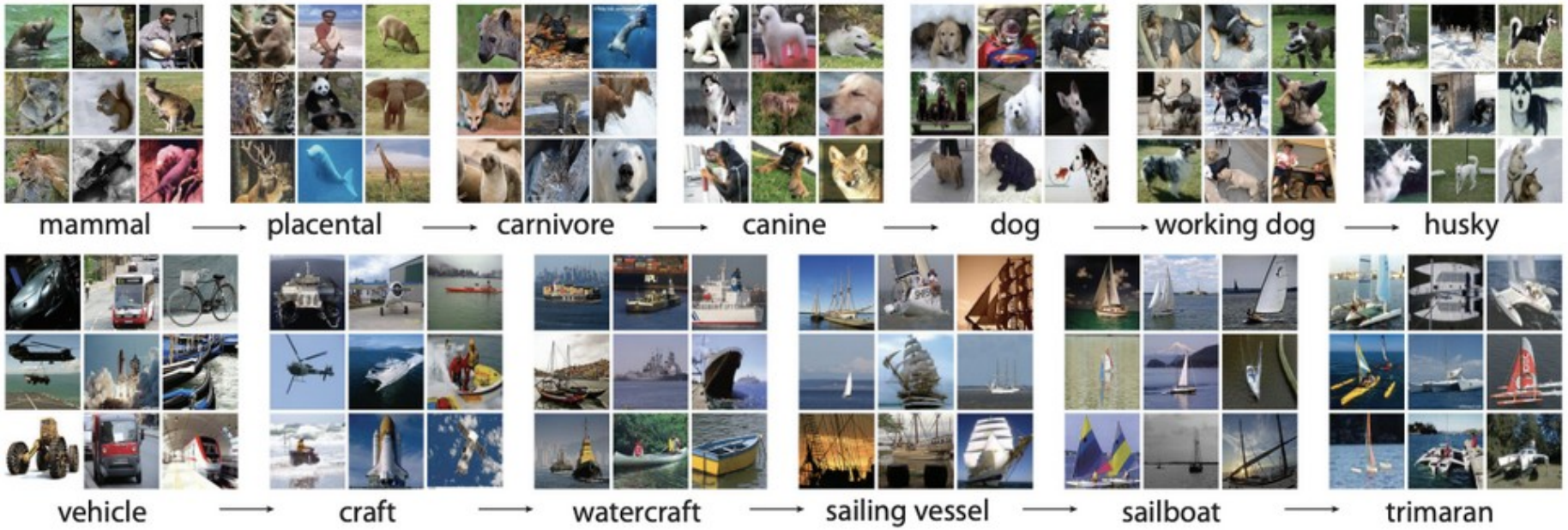


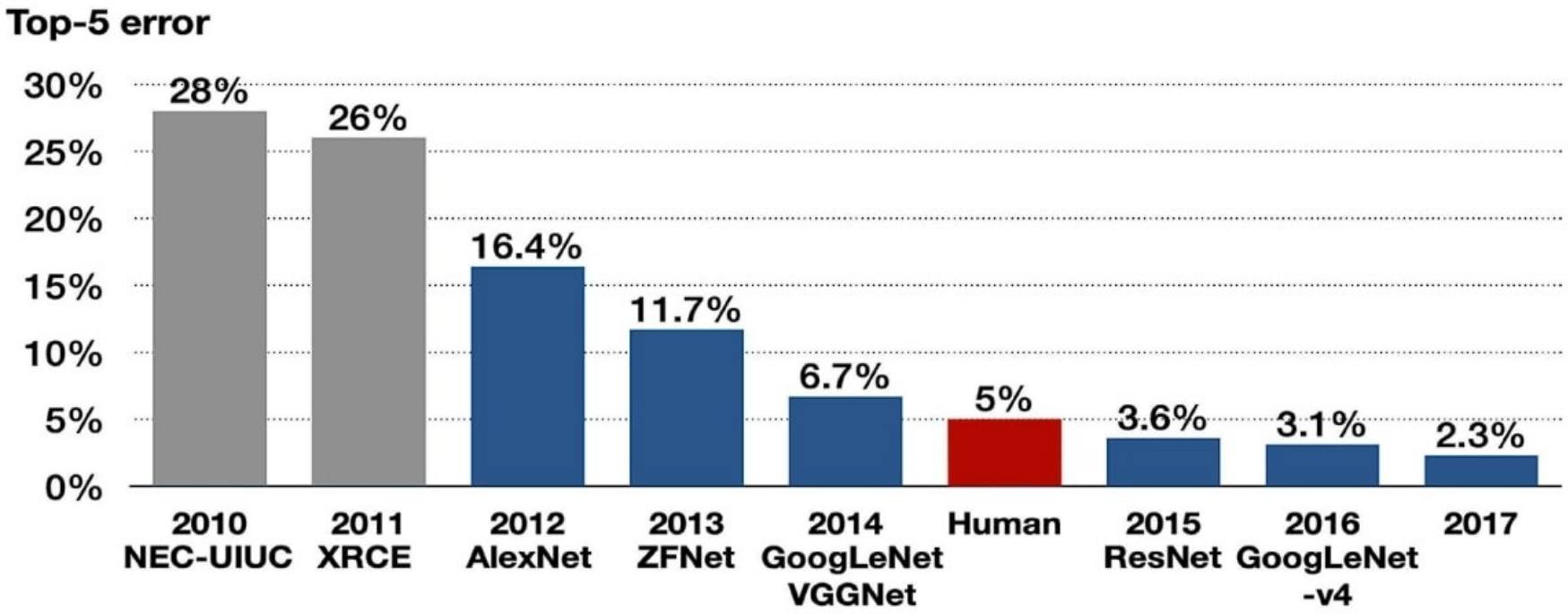
Источник: https://roboflow.com/

The **ImageNet** project is a large visual database designed for use in visual object recognition software research



ImageNet: A Large-Scale Hierarchical Image Database





Human: 5% - ошибка подготовленного аннотатора на 1500 тестовых изображений



A: Пенал

B: Линейка

C: Шариковая ручка

D: Точилка



A: Пенал

B: Линейка

C: Шариковая ручка

D: Точилка

Метка ImageNet: rule, ruler



A: Клубника

B: Апельсин

C: Камчатский краб

D: Пицца



A: Клубника

B: Апельсин

C: Камчатский краб

D: Пицца

Метка ImageNet: king crab, Alaska crab



A: Ваза

B: Маска

C: Кувшин

D: Топорик



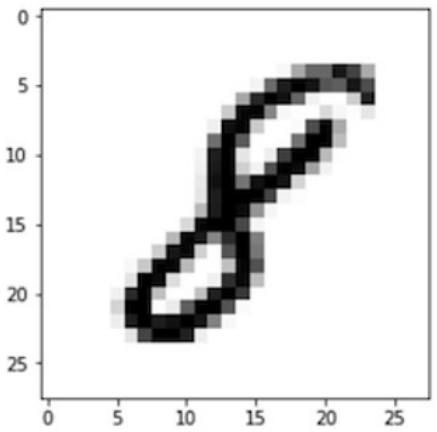
A: Ваза

B: Маска

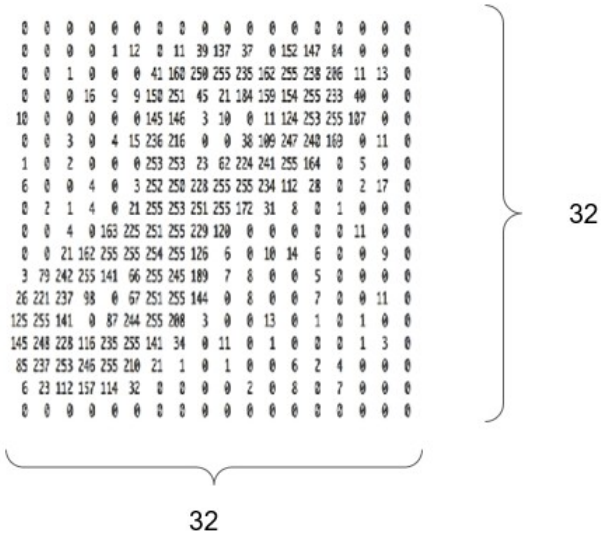
C: Кувшин

D: Топорик

Метка ImageNet: hatchet



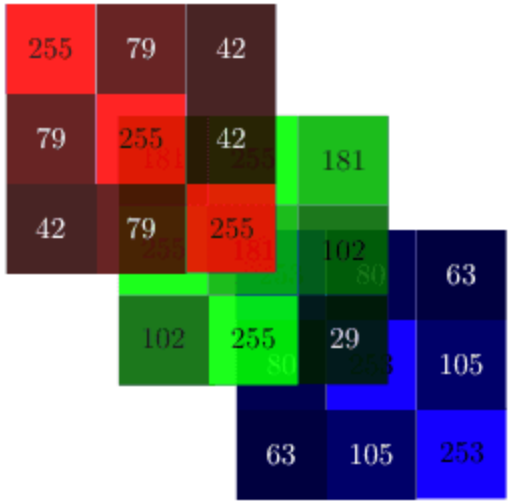
=



Grayscale

RGB





Size = 1xHxW

Size = 3xHxW

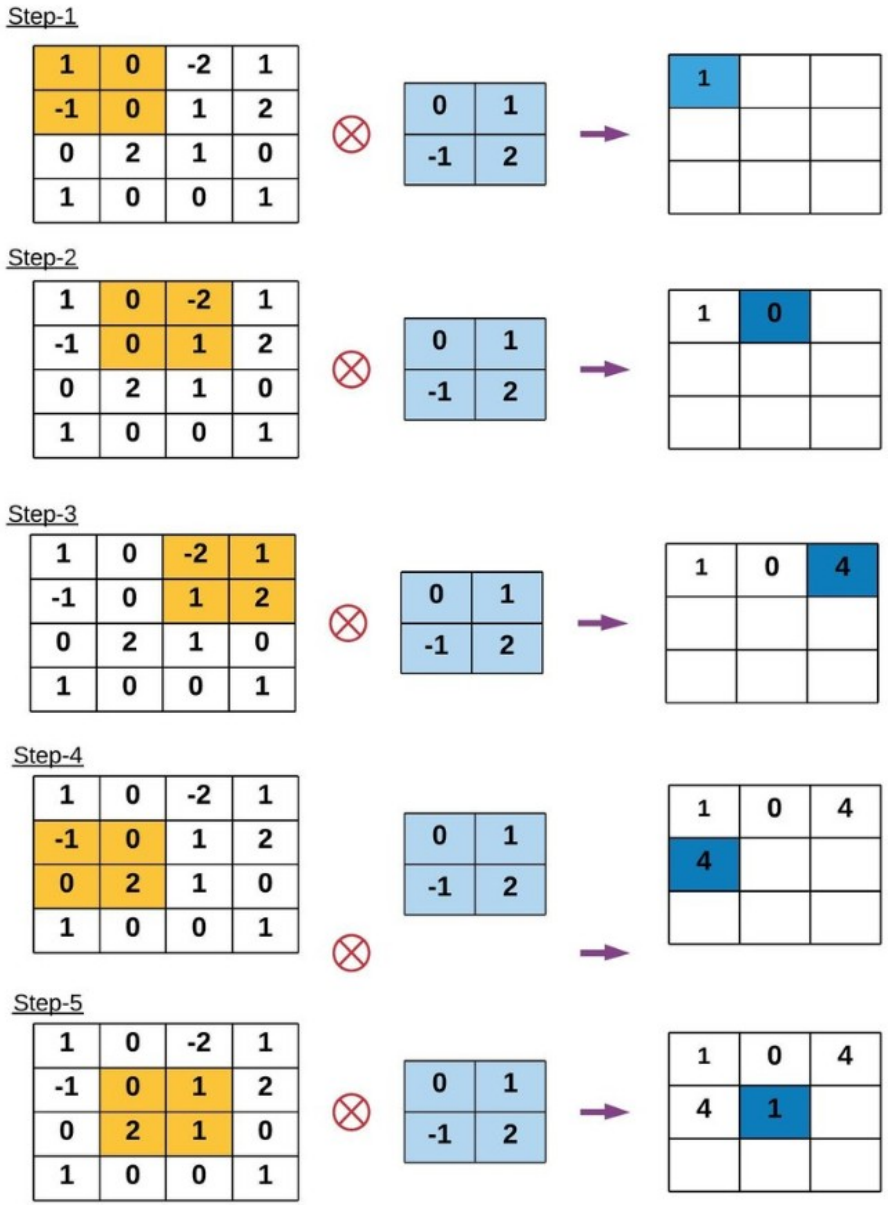
Классификация с помощью полносвязной НС с кросс-энтропийной функцией потерь:

- Явно не учитывается структура данных – хочется получить инвариантность к различным преобразованиям исходного изображения

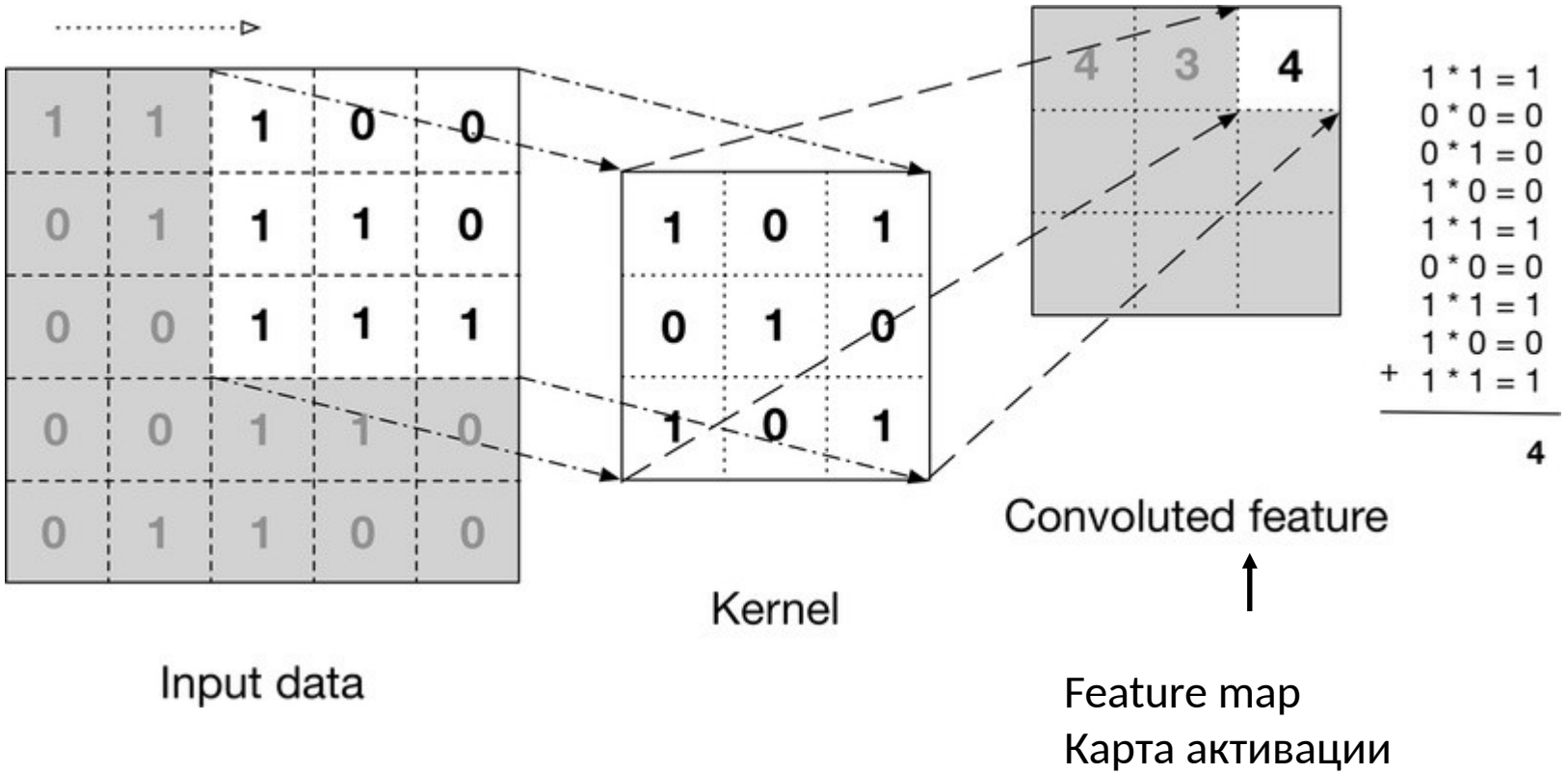


- Большое количество нейронов в первом слое сети. Например, RGB изображение размером 128x128 это вектор из 49152 элементов

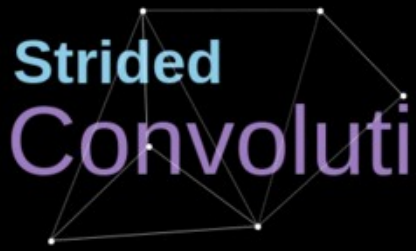
Операция свертки – поэлементное умножение и суммирование в скользящем окне



С чем сворачиваем? С ядром свертки (фильтром)



Feature map
Карта активации



Strided Convolution

Input Image

1	9	8	4	4	5	7
4	8	6	7	9	1	7
4	0	5	9	3	8	4
7	3	5	9	0	5	4
7	4	1	1	8	1	2
7	6	6	9	8	7	6
3	6	3	5	4	2	7

*

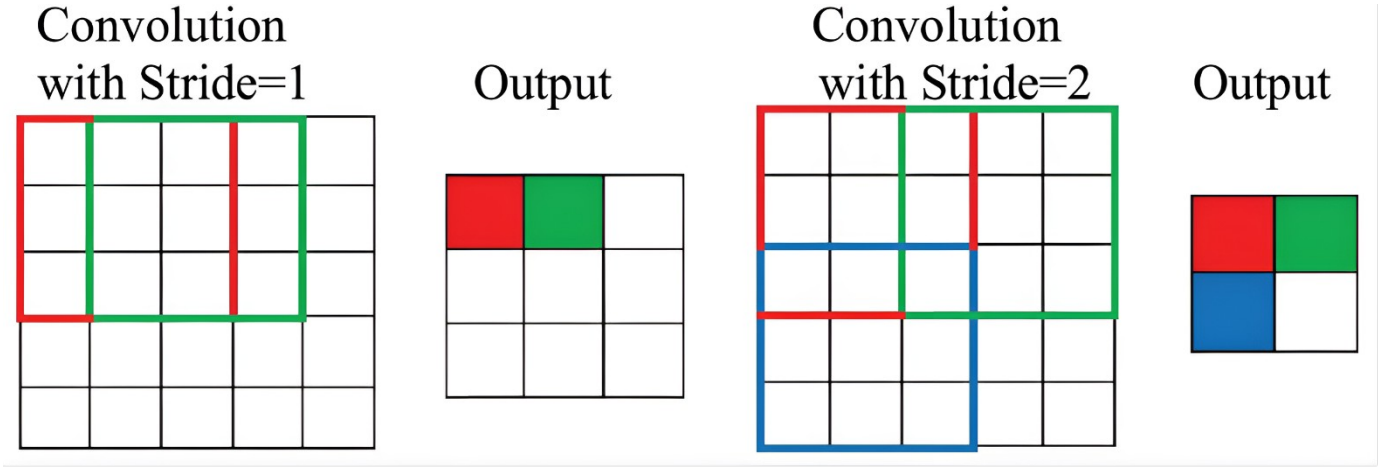
Kernel

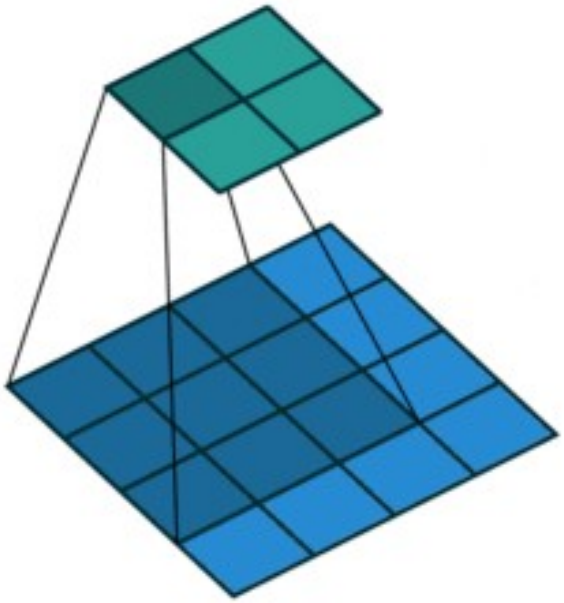
0	-1	0
-1	5	-1
0	-1	0

=

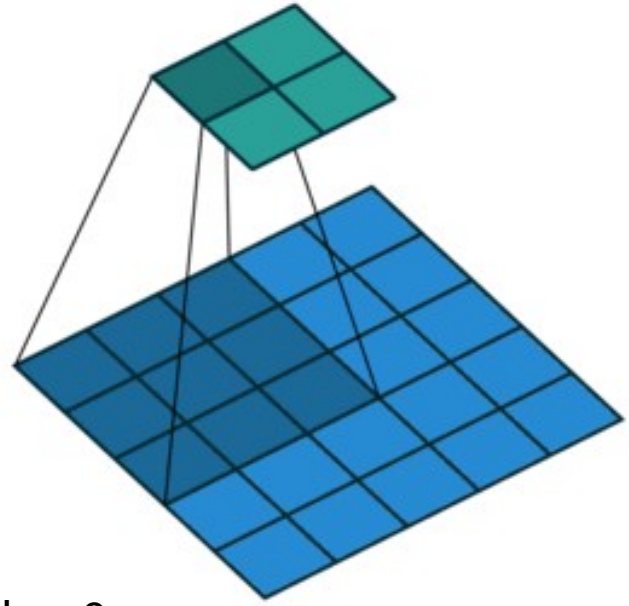
Feature Map

- Stride – величина смещения фильтра





Stride = 1



Stride = 2

- Padding – дополнение исходного изображения пикселями (рамка вокруг изображения)

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	0
0	7	8	9	10	11	12	0
0	1	2	3	4	5	6	0
0	7	8	9	10	11	12	0
0	1	2	3	4	5	6	0
0	7	8	9	10	11	12	0
0	0	0	0	0	0	0	0

Zero padding with a one-pixel thick boundary

11	10	9	10	11	12	11	10
7	6	5	6	7	8	7	6
3	2	1	2	3	4	3	2
7	6	5	6	7	8	7	6
11	10	9	10	11	12	11	10
15	14	13	14	15	16	15	14
11	10	9	10	11	12	11	10
7	6	5	6	7	8	7	6

Mirror padding with a two-pixel thick boundary

1	1	1	2	3	4	4	4
1	1	1	2	3	4	4	4
1	1	1	2	3	4	4	4
5	5	5	6	7	8	8	8
1	1	1	2	3	4	4	4
5	5	5	6	7	8	8	8
5	5	5	8	9	8	8	8
5	5	5	8	9	8	8	8

Replicate padding with a two-pixel thick boundary



zero padding



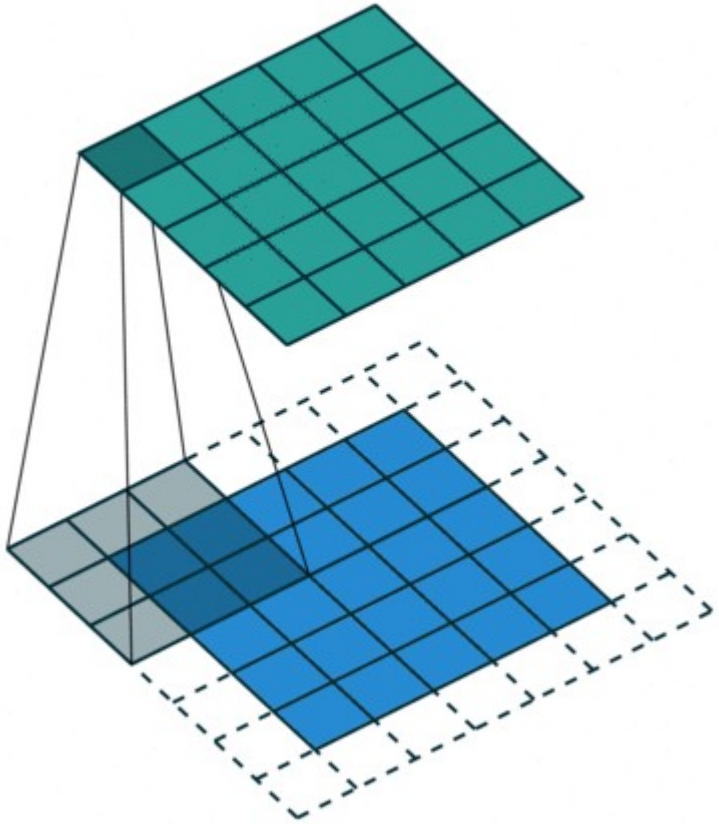
mirror padding



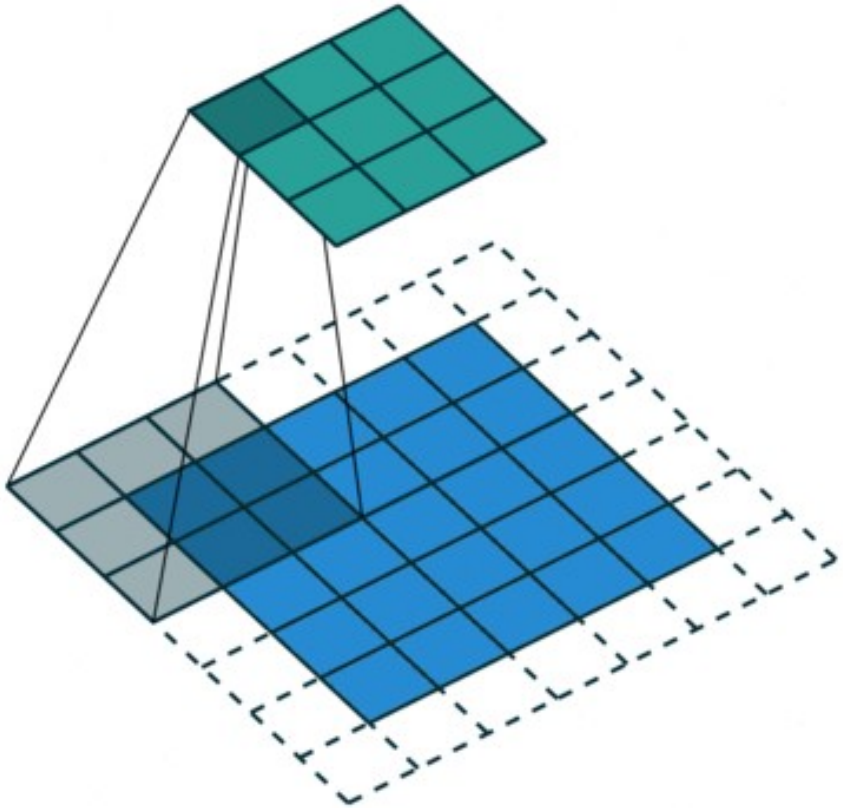
circular padding

Источник: https://habr.com/ru/articles/795223/

Padding = 1
Stride = 1



Padding = 1
Stride = 2



Источник: https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html

- Зная параметры сети (размер ядра свертки, паддинг, пуллинг и страйд) можно посчитать размер выходной матрицы признаков:

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{in}: number of input features
n_{out}: number of output features
k: convolution kernel size
p: convolution padding size
s: convolution stride size

Источник: https://syncedreview.com/2017/05/11/a-guide-to-receptive-field-arithmetic-for-convolutional-neural-networks/

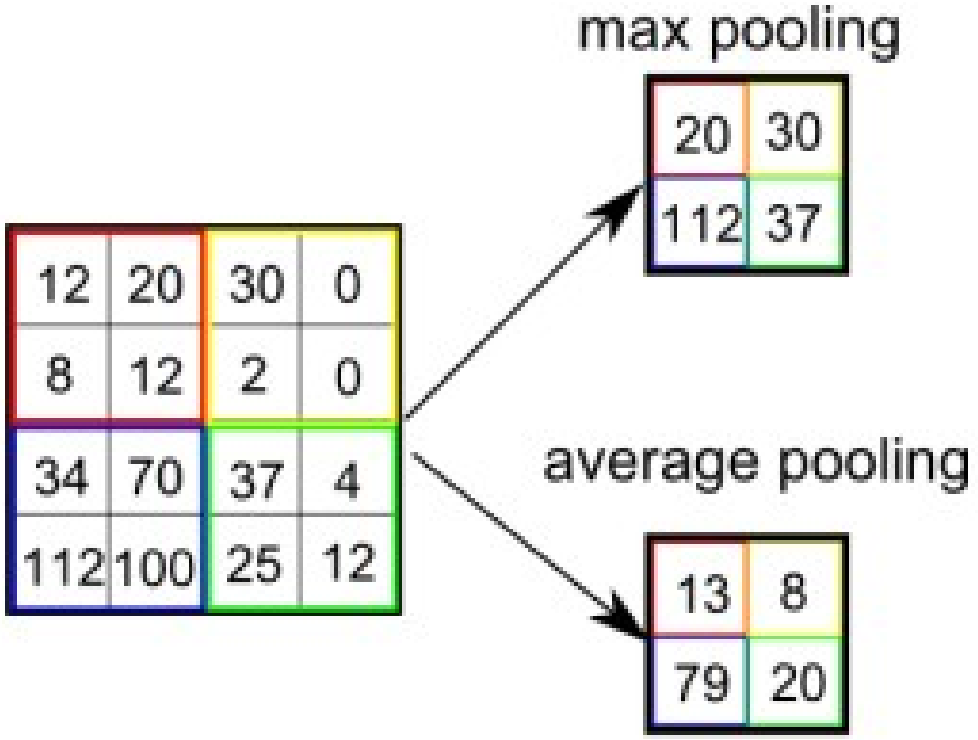
Свёртка: каналы и размеры

- Каждый фильтр охватывает все входные каналы
- Один фильтр формирует одну выходную карту, задаём число фильтров

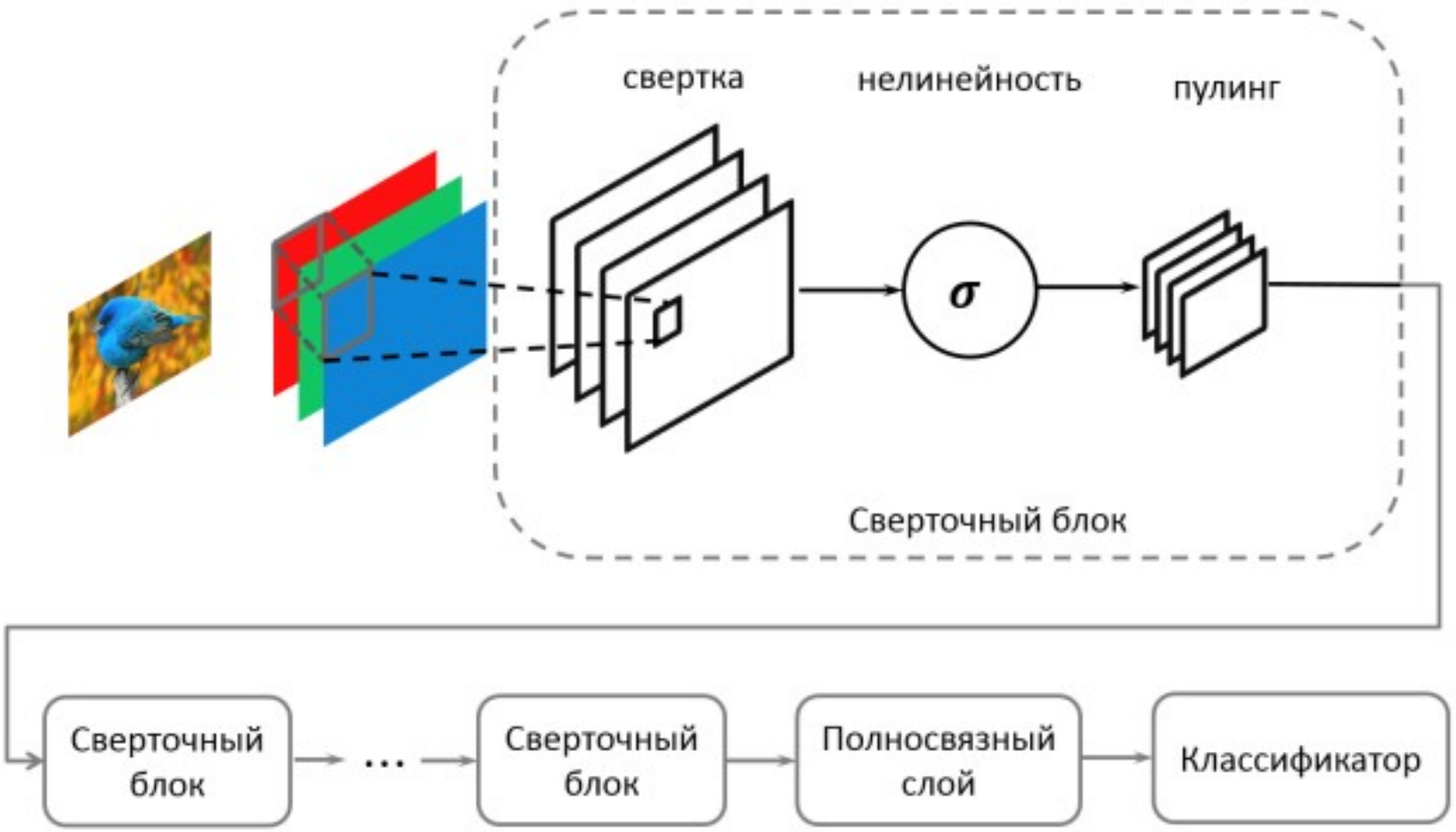
Этап	**Размер одного объекта**	**Что это означает**
RGB-изображение	3 × 32 × 32	3 входных канала
Один фильтр	3 × 3 × 3	Ядро 3 × 3 на каждом канале
16 фильтров	16 × 3 × 3 × 3	16 разных наборов весов
Выход слоя	16 × 32 × 32	16 карт признаков

Stride = 1 и padding = 1 сохраняют высоту и ширину.
Число параметров с bias: $(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$.

- Pooling – техника уменьшения размерности (downsampling) карт активации за счет агрегации значений локального окна.



Базовая схема сверточной сети



Источник: www.machinelearning.ru/wiki/images/7/71/Voron-ML-DeepLearning-slides.pdf

Фильтры “реагируют” на паттерны на изображении. Если паттерн присутствует на изображении, то карта активации после соотв. фильтра будет содержать большие числа



x

Фильтр, который
реагирует
на вертикальные линии

-1	2	-1
-1	2	-1
-1	2	-1

Активация сильнее, на карте активации большие числа

34	55	64	73	23
13	15	23	-86	-96
12	-3	0.4	71	19
11	14	17	-35	19

-1	-1	2
-1	2	-1
2	-1	-1

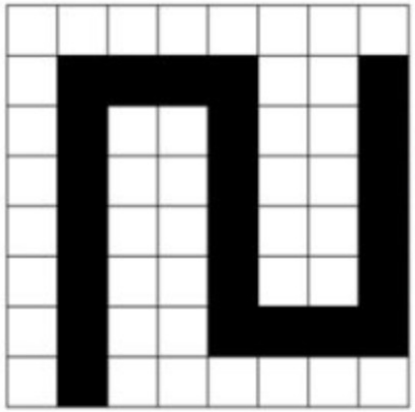
0.01	-0.2	1.8	2	-1.5
3	-0.5	6	-7	0.4
4	5	-0.8	-5	-1
1.2	0.5	3	-3	0.4

Фильтр, который
реагирует
на изогнутые линии

Активация слабее, на карте активации маленькие числа

Источник: https://www.youtube.com/watch?v=HpKGv-kYurk

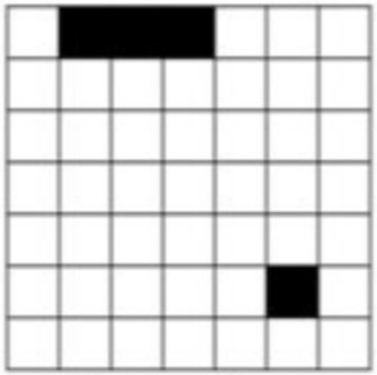
Как мог бы выглядеть фильтр, реагирующий на горизонтальные линии



+

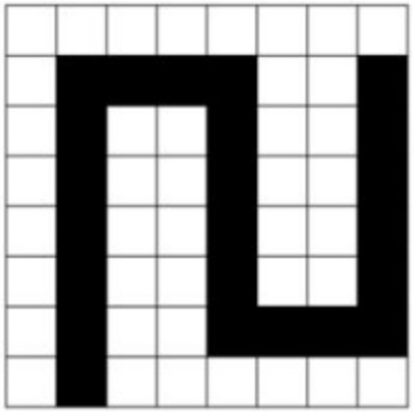
-1	-2	-1
0	0	0
1	2	1

=



Источник: https://www.youtube.com/watch?v=HpKGv-kYurk

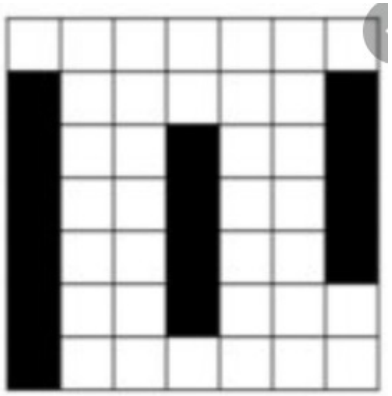
Как мог бы выглядеть фильтр, реагирующий на вертикальные линии



+

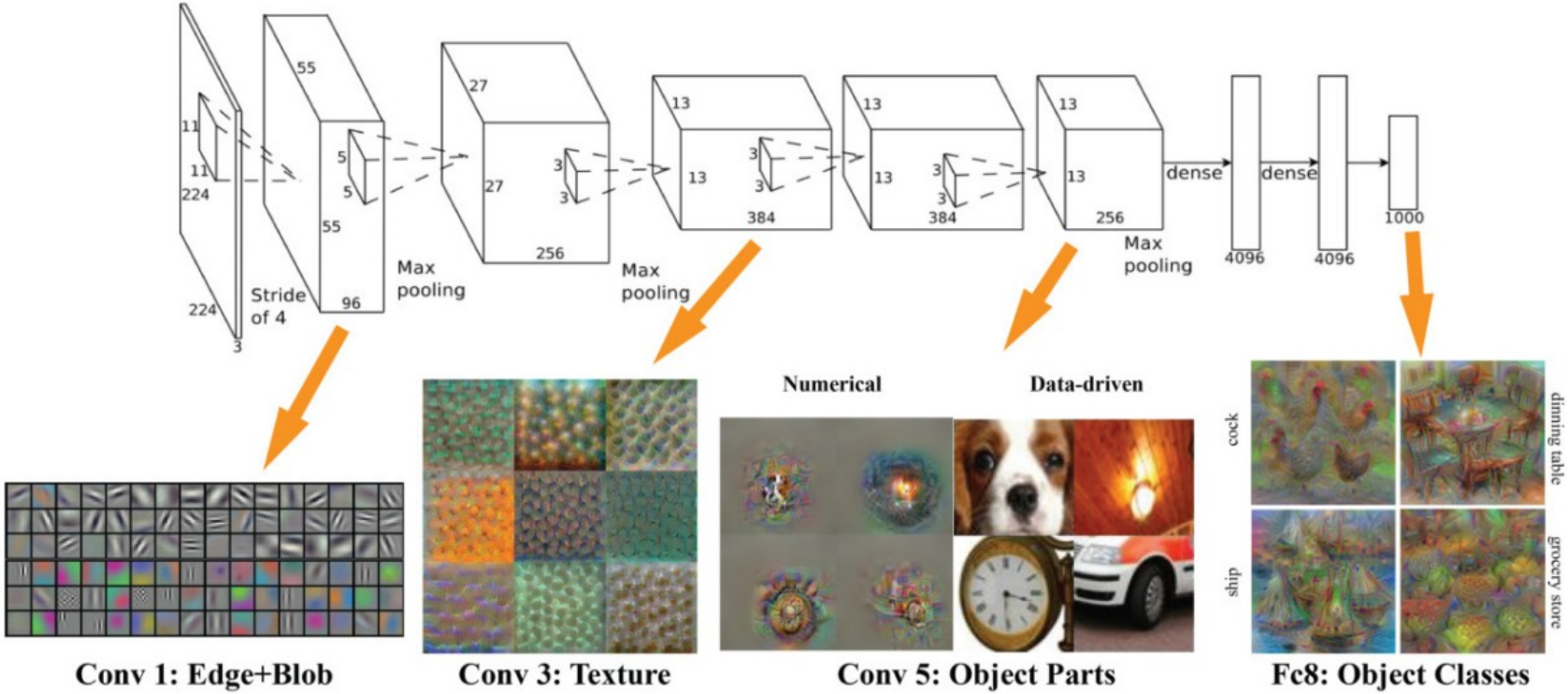
-1	0	1
-2	0	2
-1	0	1

=



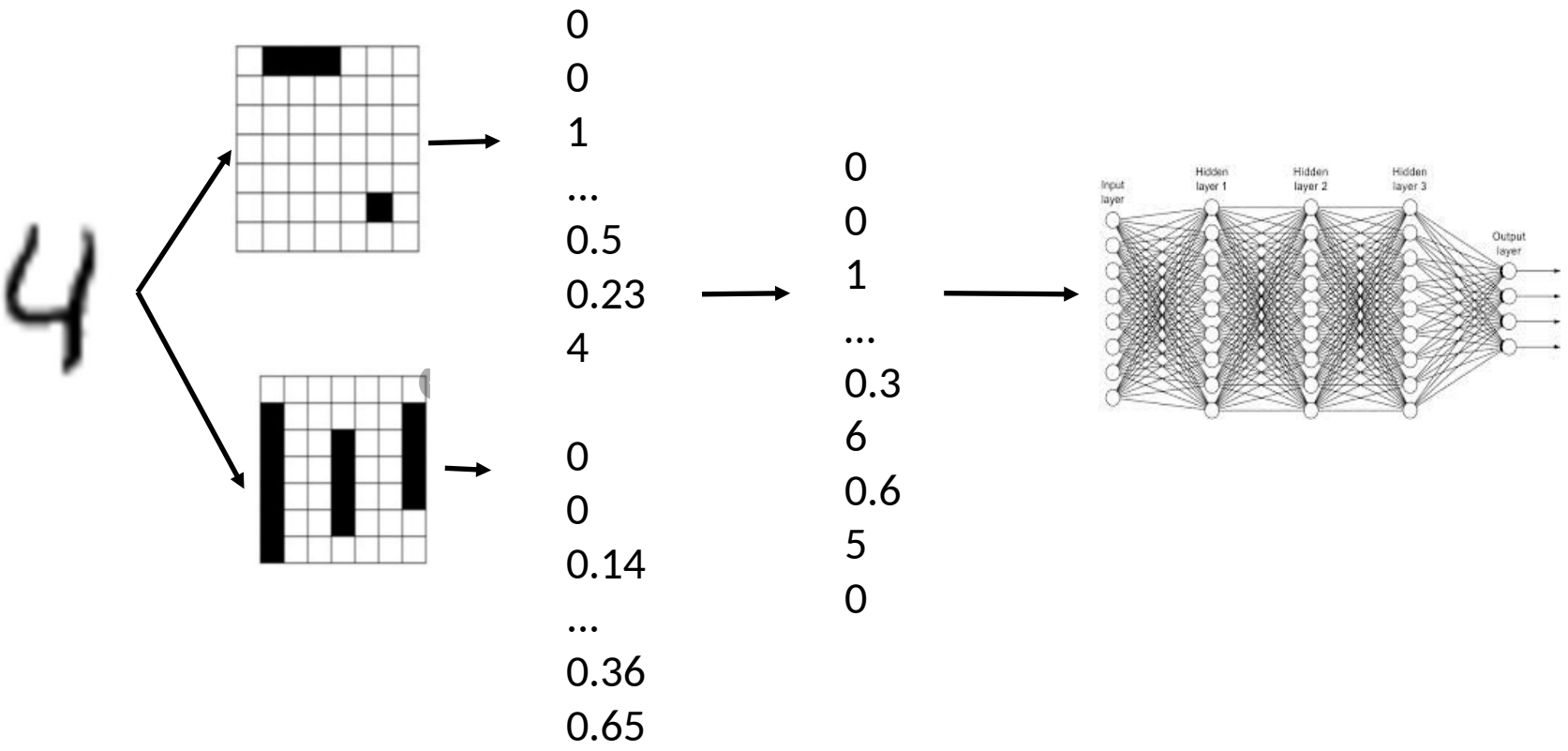
Источник: https://www.youtube.com/watch?v=HpKGv-kYurk

- Сверточная сеть обучается извлечению признаков. Чем выше слой, тем более крупные и сложные элементы изображений он способен распознавать



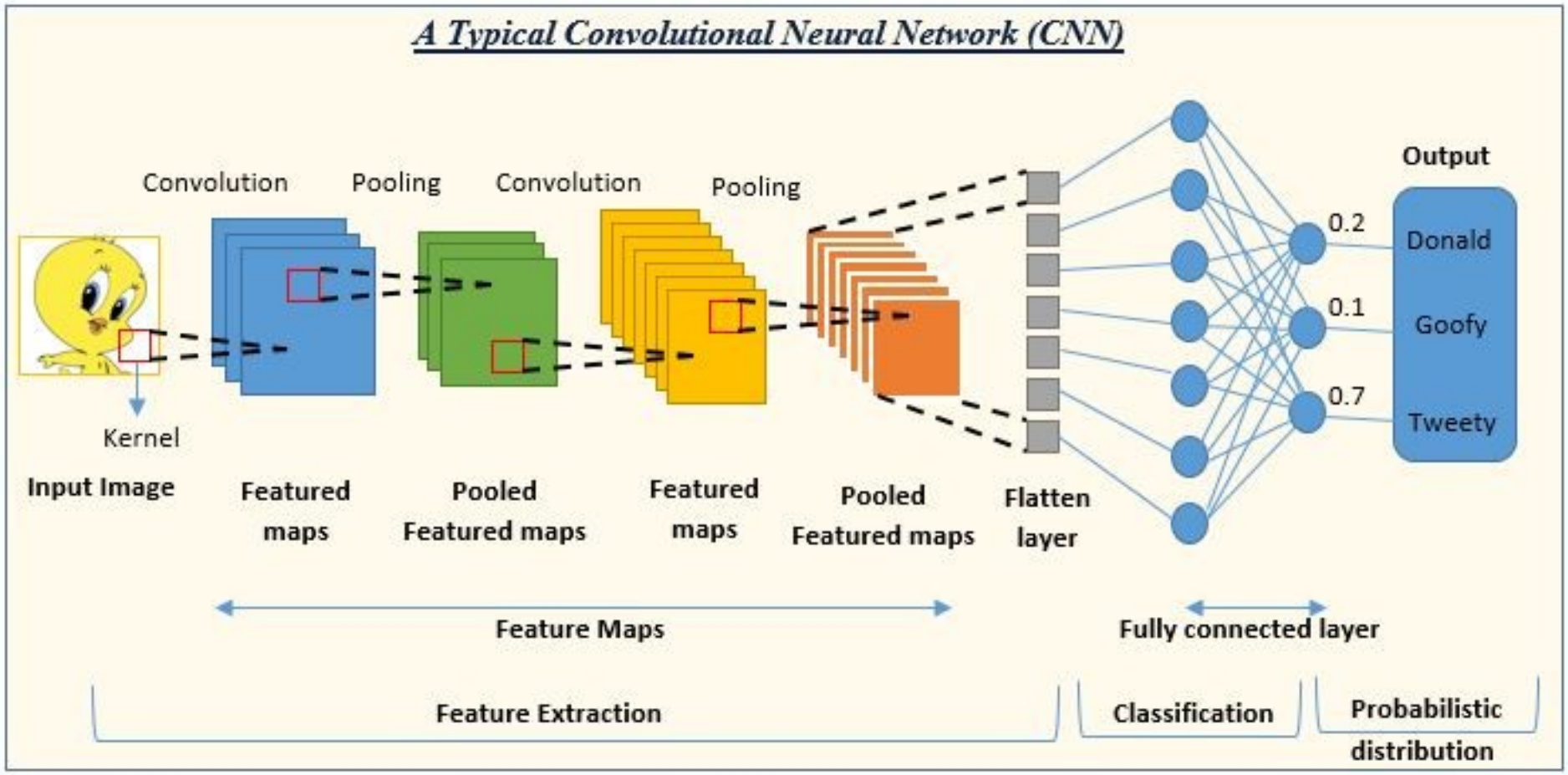
Krizhevsky A., Sutskever I., Hinton G. ImageNet classification with deep convolutional neural networks. 2012.

После получения карт активаций, мы **развернем все карты в векторы, сконкатенируем** и подадим на вход полносвязной сети

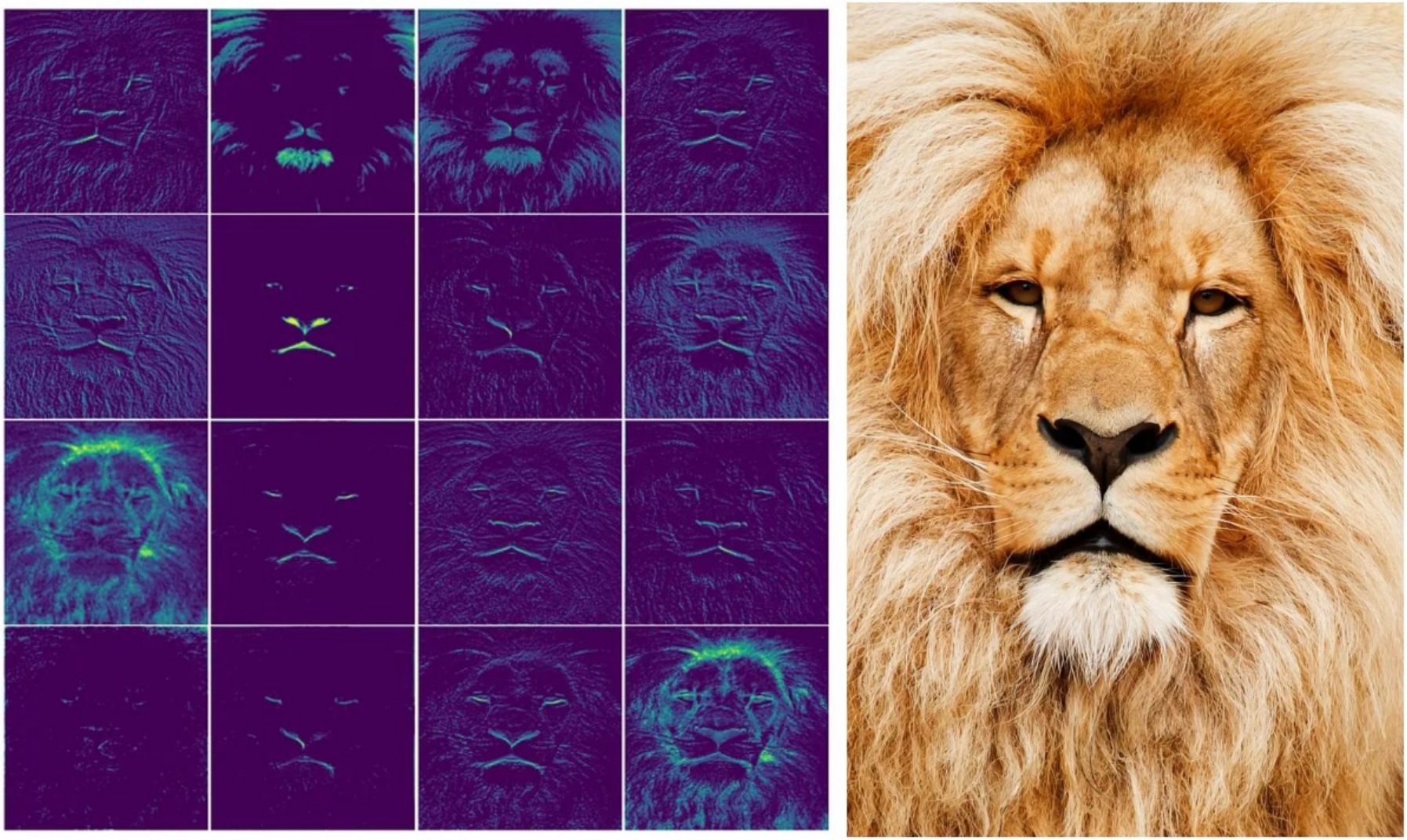


Источник: https://www.youtube.com/watch?v=HpKGv-kYurk

Вместо ручного конструирования признаков изображения с помощью операции свертки НС сама обучается извлечению признаков. Но для комплексных изображений необходимо много больше промежуточных представлений



Источник: https://ubiai.tools/the-secret-to-mastering-feature-extraction-in-convolutional-neural-network/



(**Left**) Feature extraction performed over the image of a lion using vgg19 CNN architecture (image by author). (**Right**) Original picture of the lion (public domain, availabe at Pexels).

Источник: https://towardsdatascience.com/exploring-feature-extraction-with-cnns-345125cefc9a

Свертка:

- Выявляет наличие на изображении паттерна, который задается ядром свертки
- Результат операции свертки – промежуточное представление (новое изображение)
- Чем сильнее на исходном изображении представлен паттерн, тем выше будут значения свертки
- Хотим много различных паттернов – используем много сверток

А что вообще обучать?

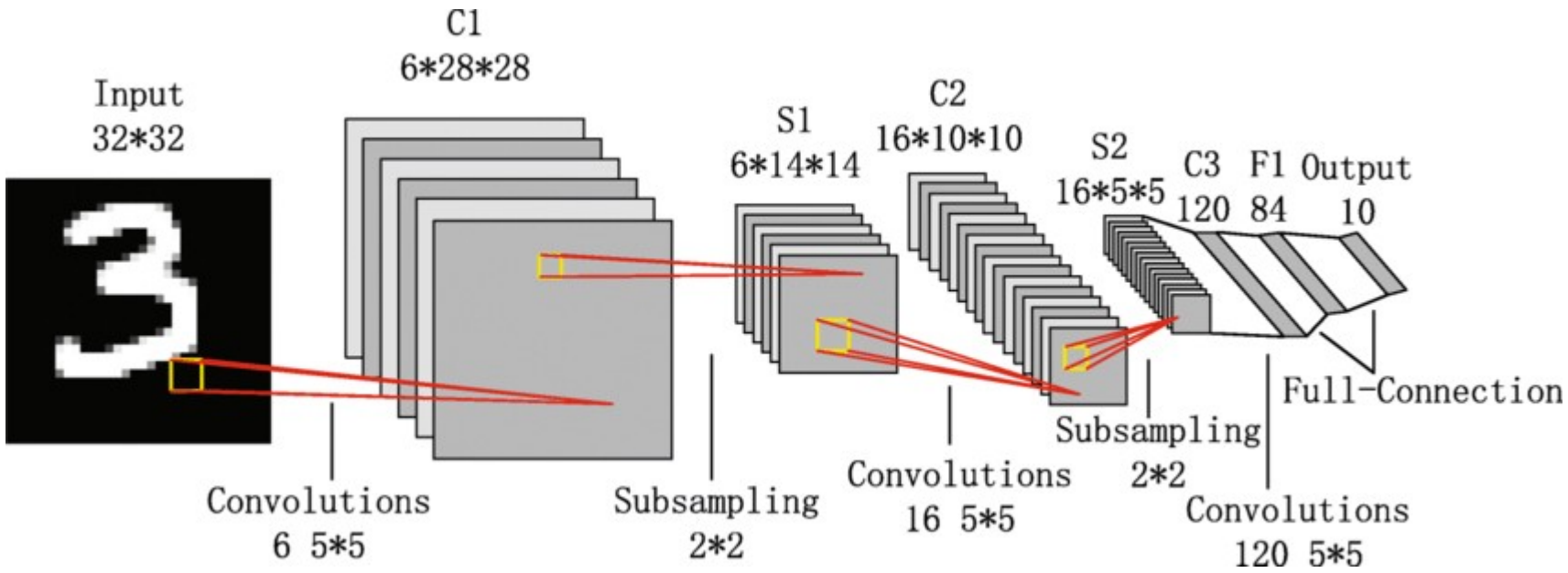
- Ядро свертки – подбирается в процессе обучения
- Веса полносвязного слоя – подбираются в процессе обучения

- Первые свертки выделяют какие-то низкоуровневые паттерны (изгибы, край, линии)

- Последние сверточные слои выделяют части изображений. Выход последнего слоя – признаковое описание изображения

- Полученные представления (признаковые описания) подаются на вход полносвязной сети. Например, для задачи многоклассовой классификации: размер выходного слоя – количество классов, функция активации – softmax –для получения вероятностей

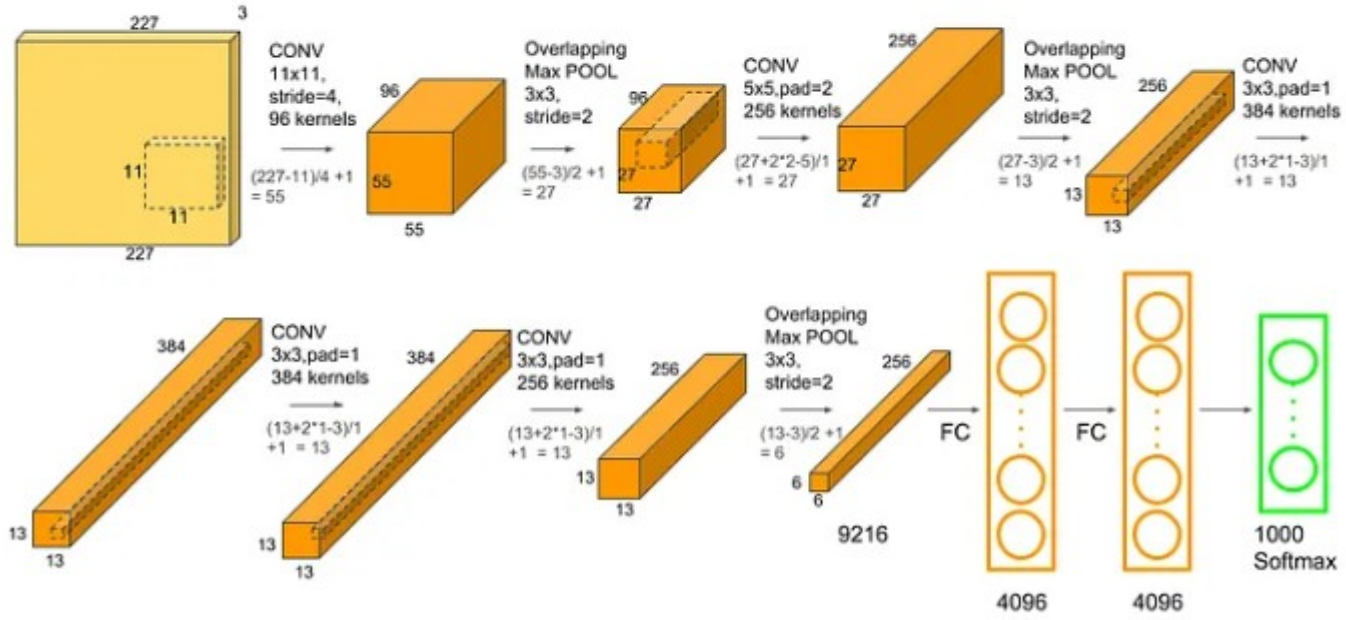
Насколько глубокие должны быть сети?



Архитектура нейронной сети LeNet-5, 1998 год. Первая версия LeNet-1 была представлена в 1989 году

Насколько глубокие должны быть сети?

- AlexNet — 2012. Uses max pooling instead of average pooling. Uses ReLU activation instead of tanh. Takes in 3 channels (RGB) as input. Obtained significantly better results in ImageNet Large Scale Visual Recognition Challenge compared to other models at the time.



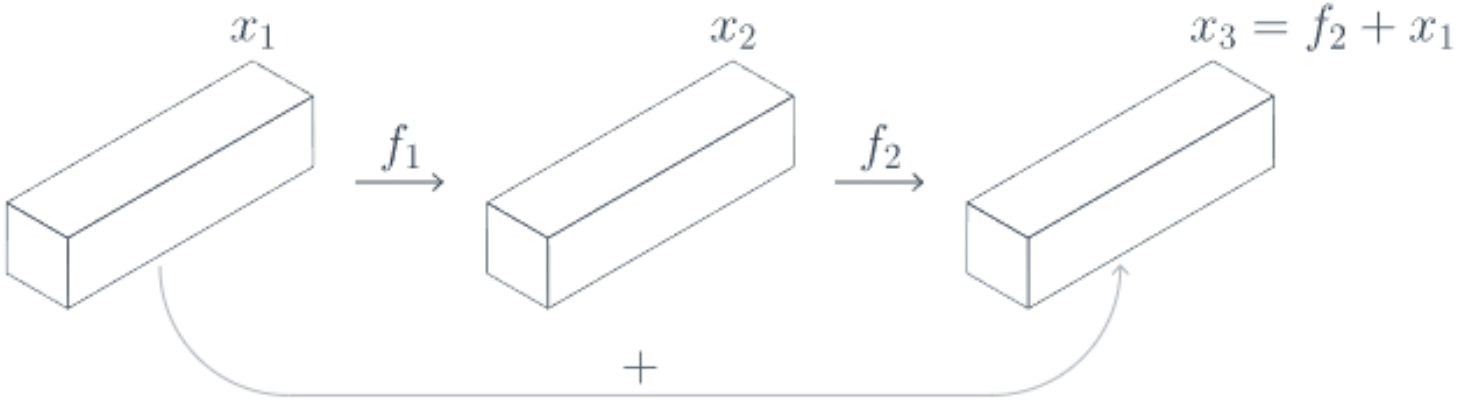
Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
out	Fully connected	__	1000	__	__	__	Softmax
F9	Fully connected	__	4096	__	__	__	ReLU
F8	Fully connected	__	4096	__	__	__	ReLU
C7	Convolution	256	13*13	3*3	1	Same	ReLU
C6	Convolution	384	13*13	3*3	1	Same	ReLU
C5	Convolution	384	13*13	3*3	1	Same	ReLU
S4	Max Pooling	256	13*13	3*3	2	Valid	__
C3	Convolution	256	27*27	5*5	1	Same	ReLU
S2	Max Pooling	96	27*27	3*3	2	Valid	__
C1	Convolution	96	55*55	11*11	4	Valid	ReLU
In	Input	3(RGB)	227*227	__	__	__	__

AlexNet Summary

- Чем больше сверток, тем лучше?

Увеличение глубины может приводить к усложнению оптимизации и затуханию градиента.

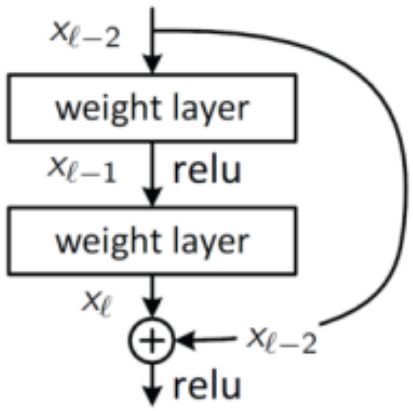
- Residual connection (архитектура ResNet, Residual NN) – прокидываем признаки на предыдущем слое мимо сверток на следующем, что облегчает обучение



- DenseNet – вместо суммы конкатенация, может улучшать результат, но вычислительно сложно

Сквозная связь (skip connection) слоя ℓ с предшествующим слоем $\ell - d$:

$$x_\ell = \sigma(Wx_{\ell-1}) + x_{\ell-d}$$



Слой ℓ выучивает не новое векторное представление x_ℓ, а его приращение $x_\ell - x_{\ell-d}$

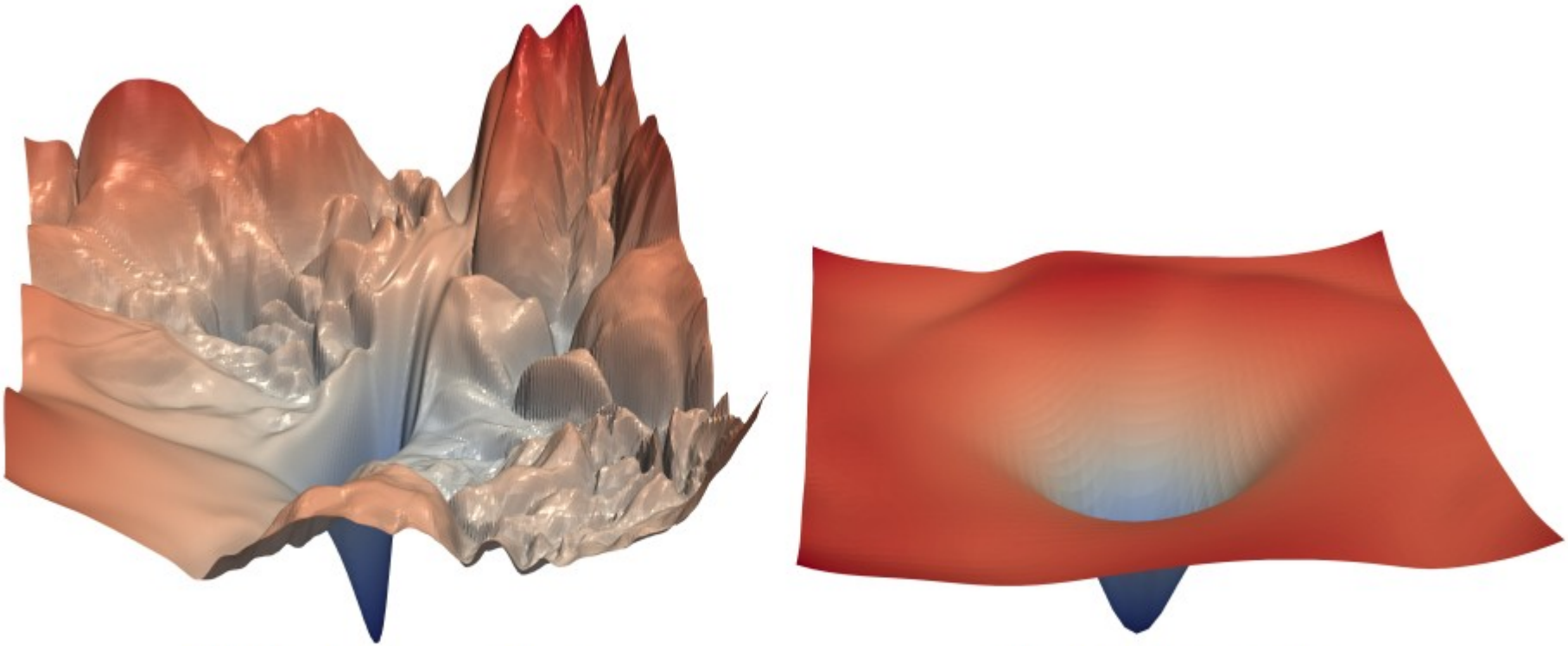
- Приращения более устойчивы $\Rightarrow$ улучшается сходимость
- Появляется возможность увеличивать число слоёв
- Обобщение — Highway Networks:

$$x_\ell = \sigma(Wx_{\ell-1})\underbrace{\tau(W'x_{\ell-1})}_{\text{transform gate}} + x_{\ell-d}\underbrace{\big(1-\tau(W'x_{\ell-1})\big)}_{\text{carry gate}}$$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Deep Residual Learning for Image Recognition. 2015

R.K.Srivastava, K.Greff, J.Schmidhuber. Highway Networks. 2015

Источник: http://www.machinelearning.ru/wiki/images/7/71/Voron-ML-DeepLearning-slides.pdf



(a) without skip connections

(b) with skip connections

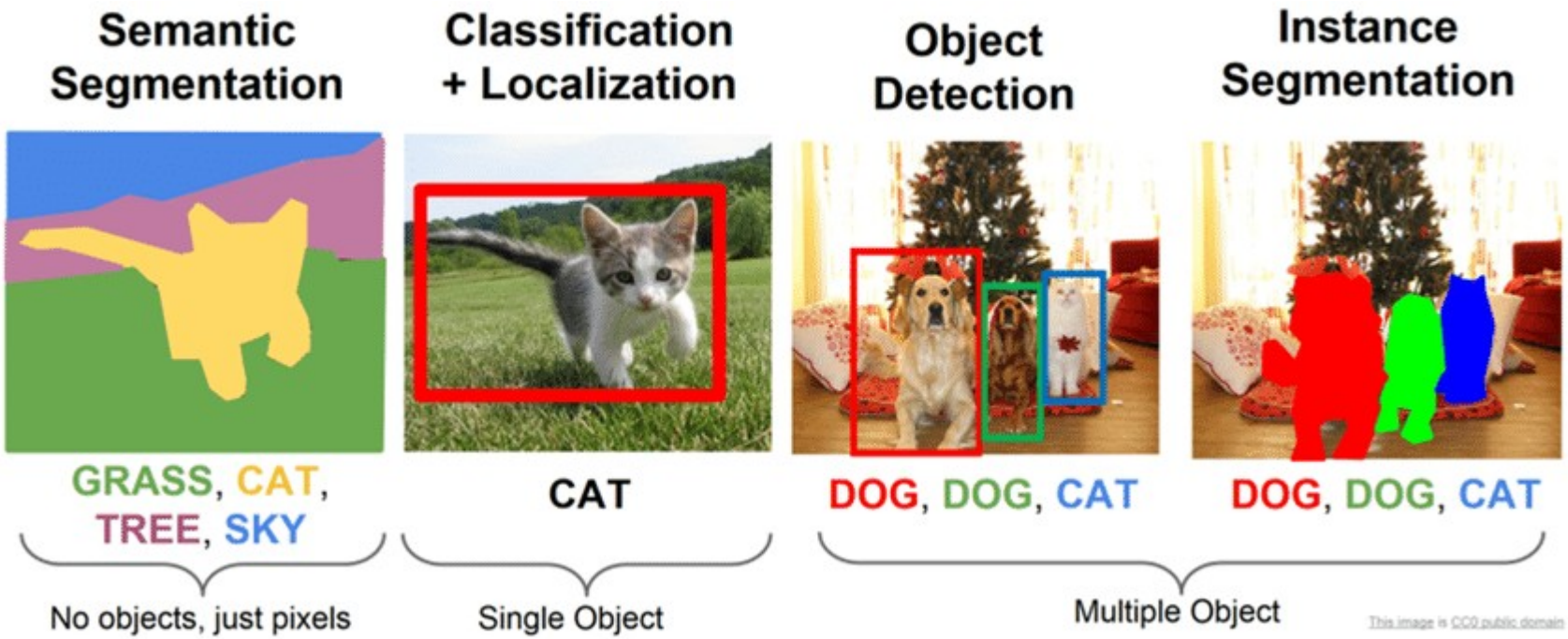
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Источник: https://arxiv.org/pdf/1712.09913

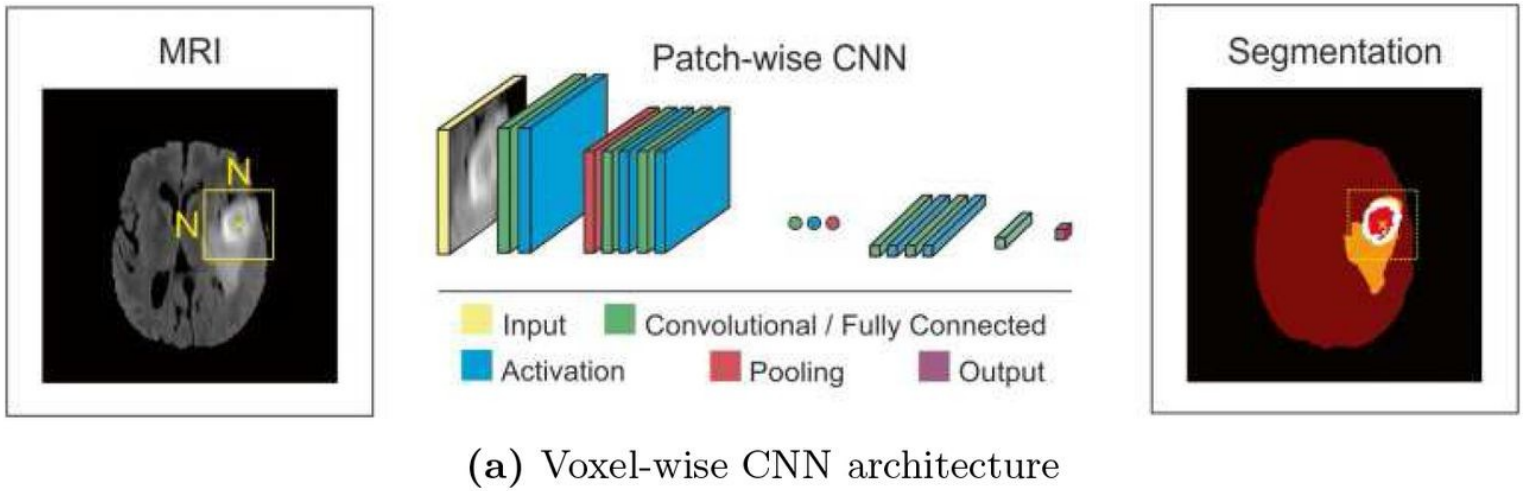
Что учитываем при обучении CNN:

- Функции активации ReLU
- Адаптивные градиентные методы
- Dropout
- Batch normalization
- Residual NN
- Подбираем число слоев и размеры слоев
- Используем аугментацию

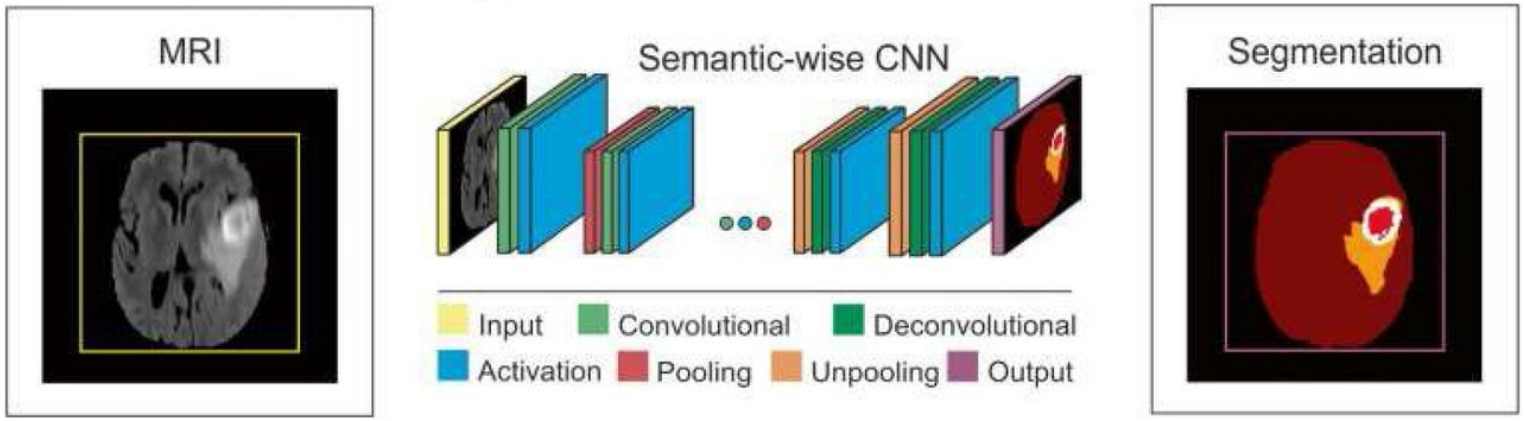
Практическое применение CNN



Применение в медицине



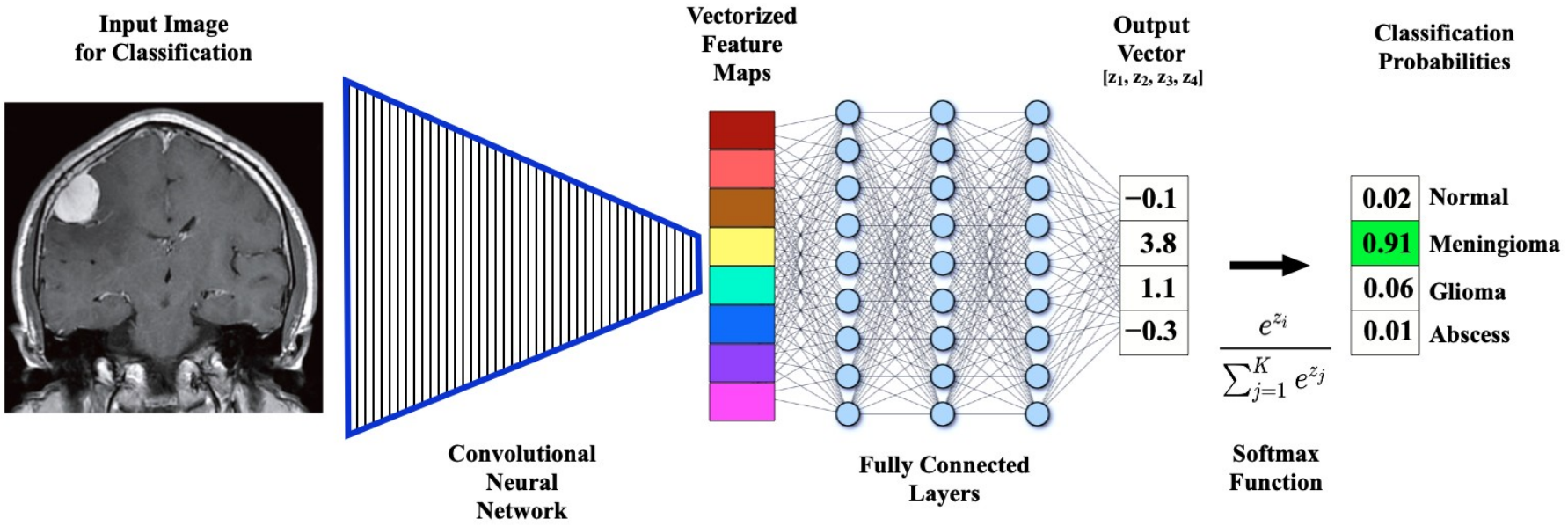
(a) Voxel-wise CNN architecture



(b) Semantic-wise CNN architecture

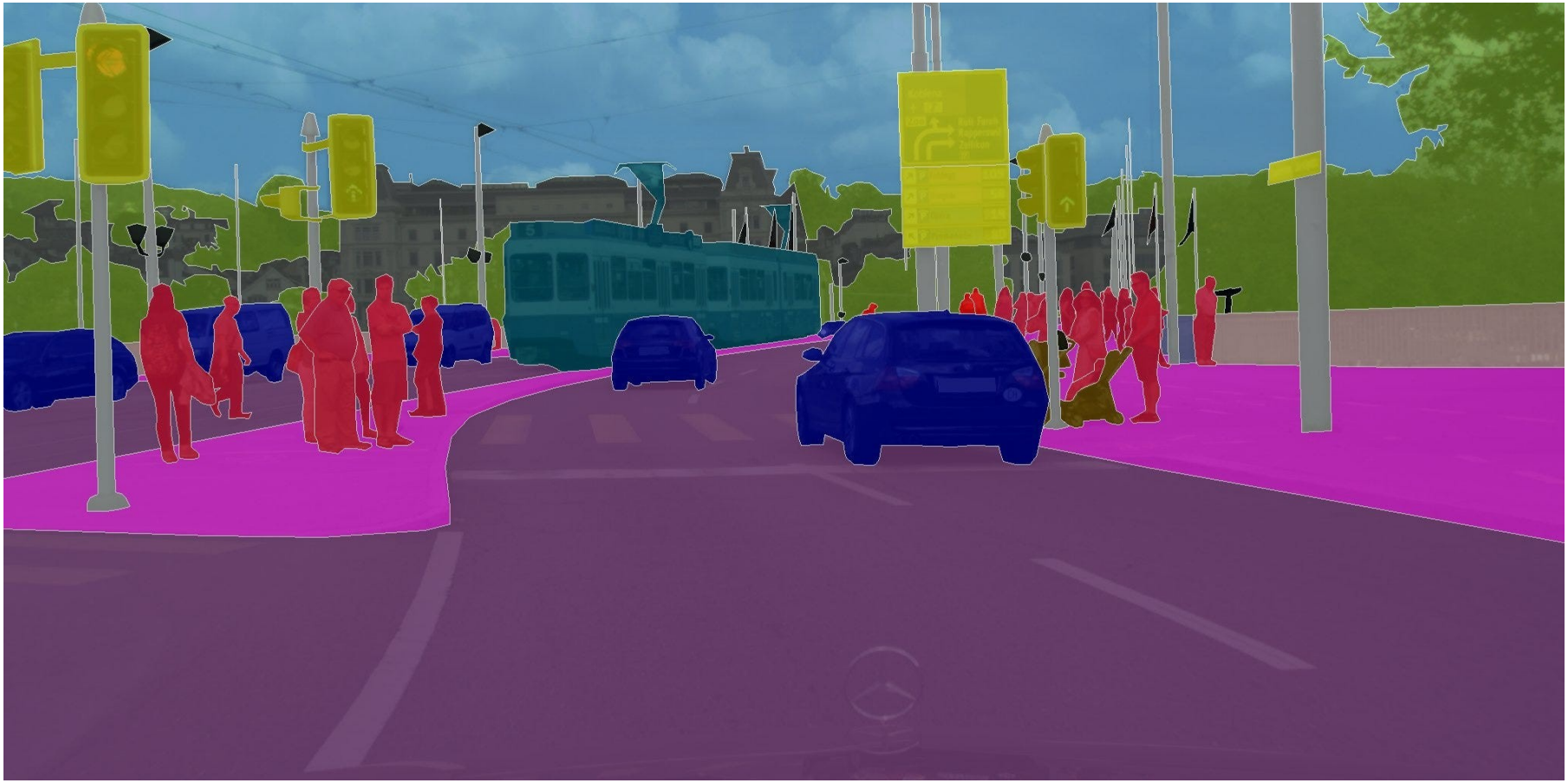
Источник: https://www.biorxiv.org/content/10.1101/2020.01.29.924985v1.full

Применение в медицине



Источник: https://mriquestions.com/convolutional-network.html#/

Применение в беспилотном транспорте

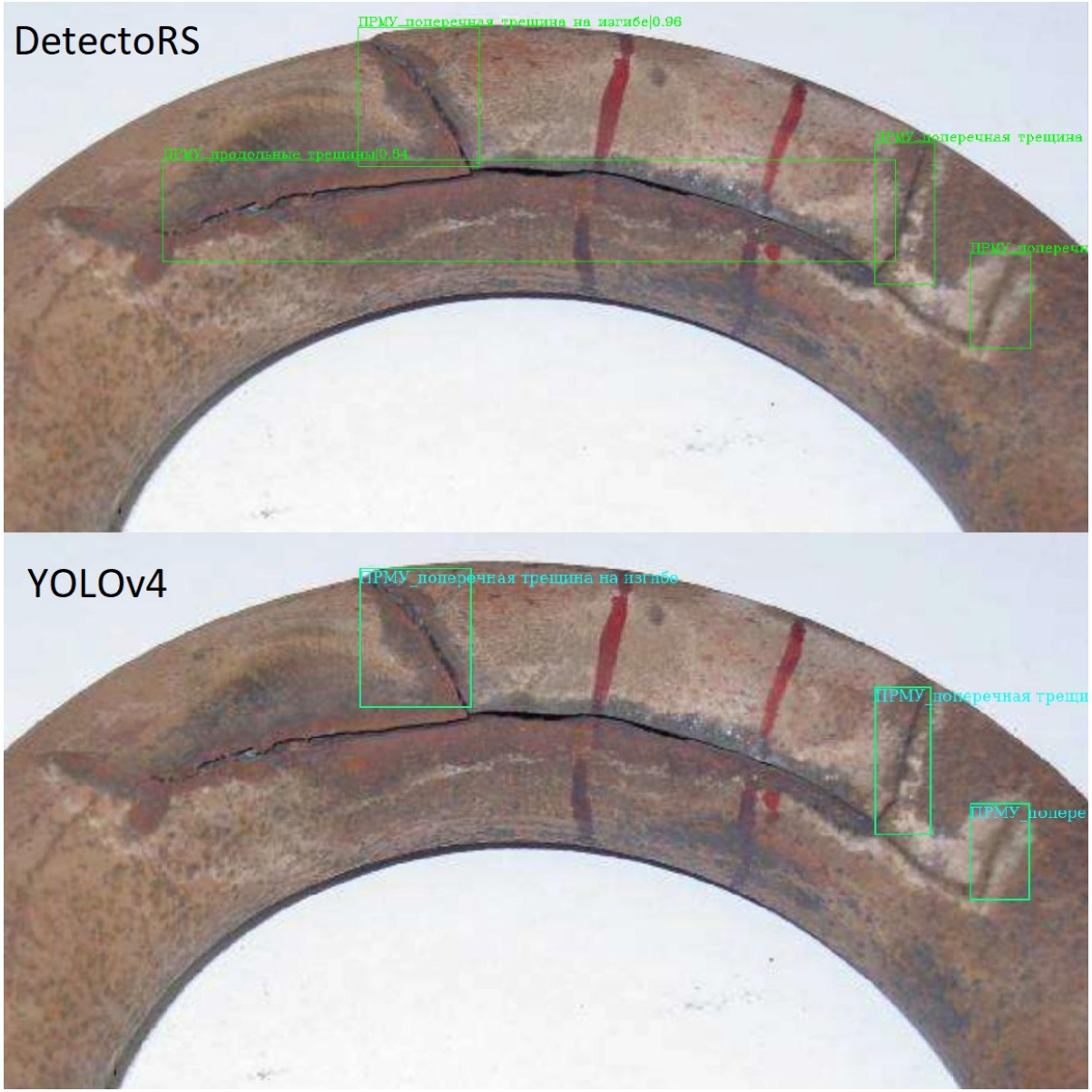


Источник: https://medium.com/analytics-vidhya/15-best-open-source-autonomous-driving-datasets-34324676c8d7

Применение в беспилотном транспорте



Применение в промышленности (поиск дефектов)



Источник: https://habr.com/ru/companies/itmai/articles/541858/

В прикладных задачах CV в качестве бэйзлайна или даже готового решения часто используются (пред)обученные модели.
Готовые веса проверяем на своих данных, если качества недостаточно – дообучаем.

Классификация и признаки изображения

Модели	**Тип модели**	**Функционал «из коробки»**	**Когда дообучаем**
ResNet / ResNeXt ConvNeXt V1 / V2	Классификатор / экстрактор признаков	Известный класс или признаковое представление	Новые классы, домен, особенности данных
MobileNet V2 / V3 EfficientNet V1 / V2	Классификатор / экстрактор признаков	Классификация, выбор размера сети под устройство	Новые классы, домен, особенности данных
ViT / DeiT Swin V1 / V2	Визуальный энкодер / классификатор	Классификация известных категорий или извлечение признаков изображения	Новые классы, недостаток качества на своих данных
DINO V2 / V3	Визуальный энкодер	Извлечение признаков для поиска похожих изображений и обучения классификатора	Для своих классов обучаем классификатор на готовых признаках, можно дообучить энкодер
CLIP / SigLIP V1 / V2	Модель сопоставления изображений и текста	Поиск изображений по описанию, классификация по заданным текстовым описаниям классов без дообучения	Специфические объекты плохо различаются готовой моделью

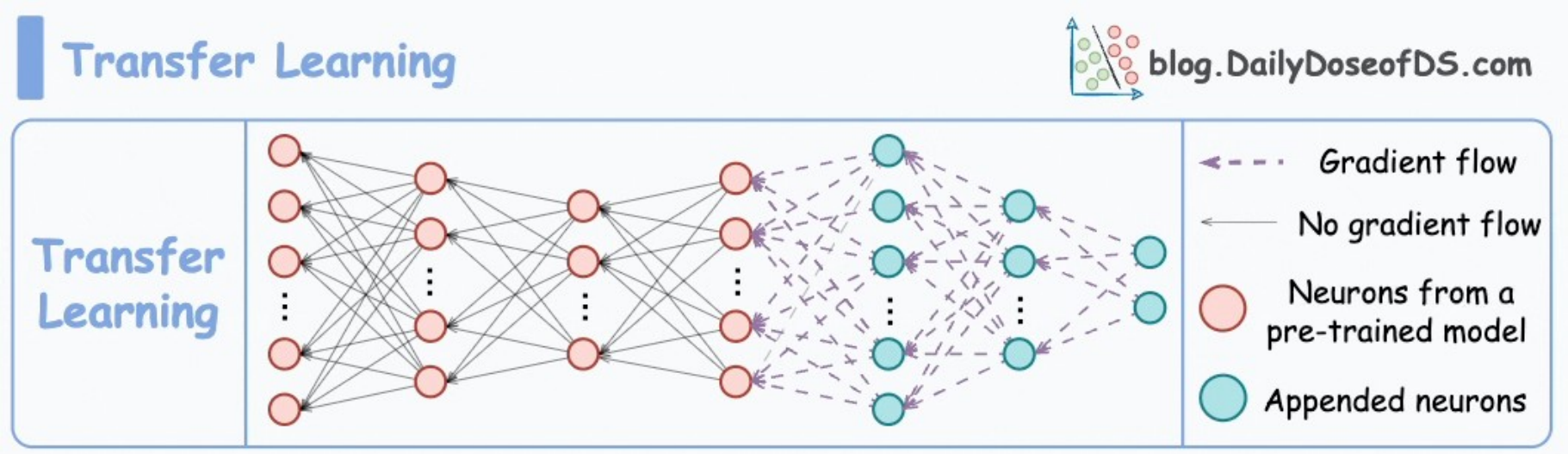
Обнаружение объектов, выделение масок и ключевых точек

Модели	**Тип модели**	**Готовые возможности**	**Когда дообучать**
YOLOv8 / YOLO11 (разные варианты)	Модели разных задач[1]	Класс, рамки, маски, ключевые точки, повёрнутые рамки	Свои классы, объекты, ключевые точки или другой домен
RT-DETR / RT-DETRv2 Faster R-CNN	Детектор	Рамки и классы известных объектов	Свои объекты или условия съёмки
Grounding DINO OWLv2	Детектор по текстовому запросу	Рамки объектов, заданных описанием	Недостаточное качество на специфических объектах
DeepLabV3 / V3+ SegFormer	Семантический сегментатор	Класс каждого пикселя для известных классов	Новые классы или домен, нужны пиксельные маски
Mask R-CNN Mask2Former[2]	Сегментатор экземпляров	Отдельные маски и классы объектов	Свои объекты, нужны маски экземпляров
SAM / SAM 2 / SAM 2.1	Сегментация по подсказке	Маска по точке / рамке. SAM 2: также видео	Готовые маски неточны в вашем домене

[1] YOLO: детектор, классификатор, instance-сегментатор, pose-estimator и OBB-детектор.
Под задачу выбираем отдельные веса.
[2] Mask2Former: также semantic / panoptic.

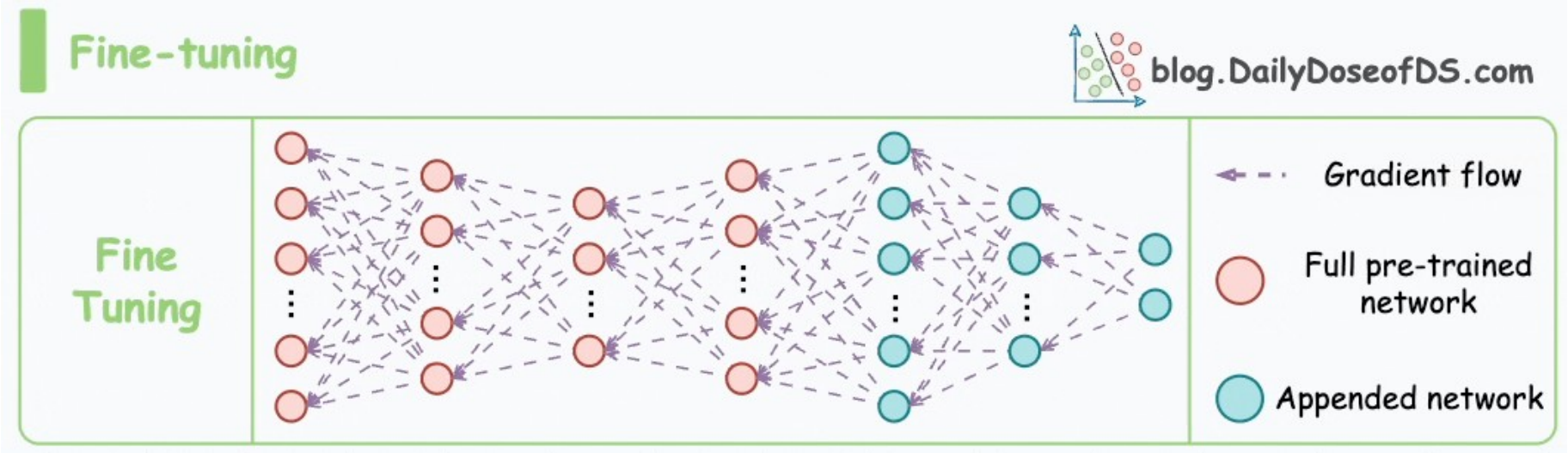
Transfer learning

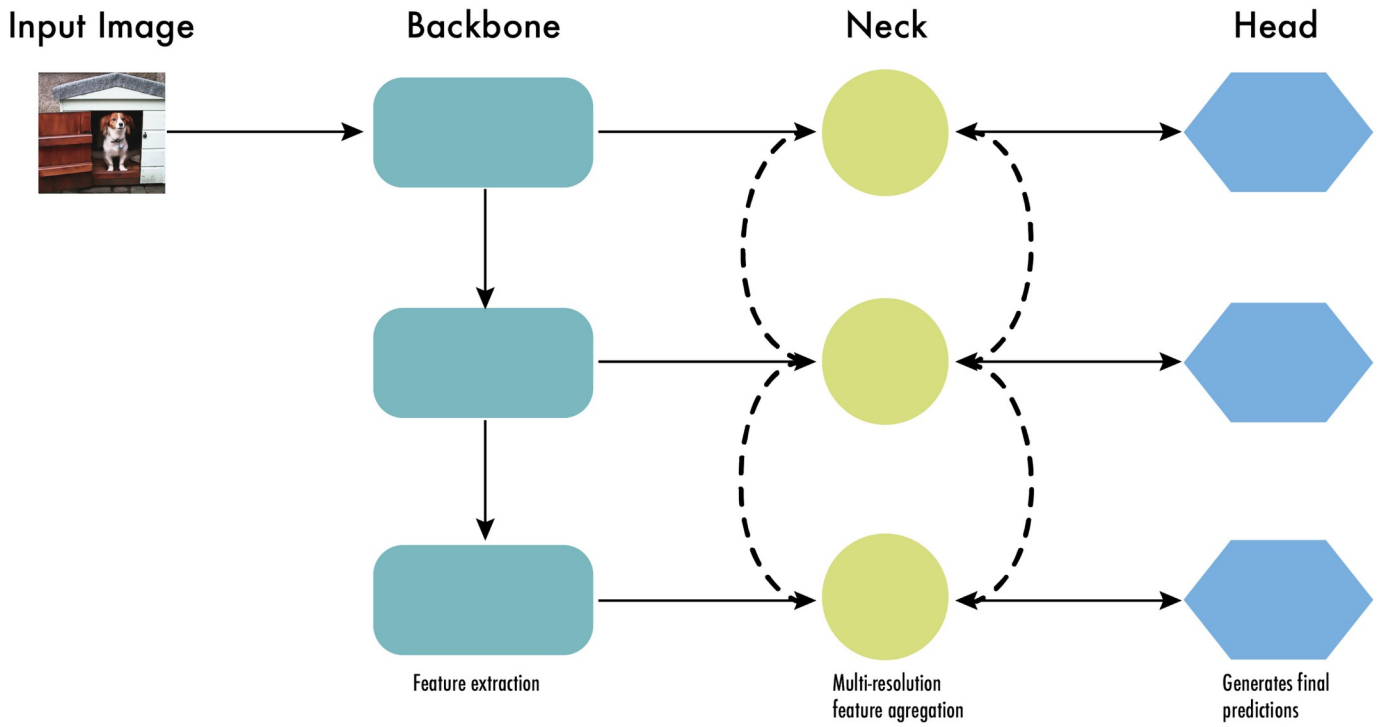
- Перенос обучения (transfer learning) – метод обучения, при котором модель, обученная для одной задачи (связанная задача), переиспользуется для схожей задачи (целевая задача)
- Целевая задача содержит мало данных. Например, классификаця специфических изображений. Связанная содержит много данных, есть обученная модель которая хорошо решает связанную задачу.
- Заменяем от 1 до N последних слоев, обучаем только эти слои (веса в остальных слоях не изменяются)



Fine-tuning

- Fine tuning – метод дополнительного обучения сети, при котором обученная на схожей задаче модель обучается на новом наборе данных. При этом веса не инициализируются случайно, а используются уже оптимизированные в процессе предыдущего обучения веса нейронной сети
- Можно добавить несколько слоев на выходе нейронной сети
- Дообучается часть или все слои





The architecture of modern object detectors: high-level backbone, neck, and head diagram

Backbone:
извлечение признаков

CNN: карты признаков
ViT: последовательность векторов

Neck
Обработка признаков

Объединяет и уточняет признаки разных масштабов.
Отдельный neck необязателен

Head (голова)
Предсказание ответа

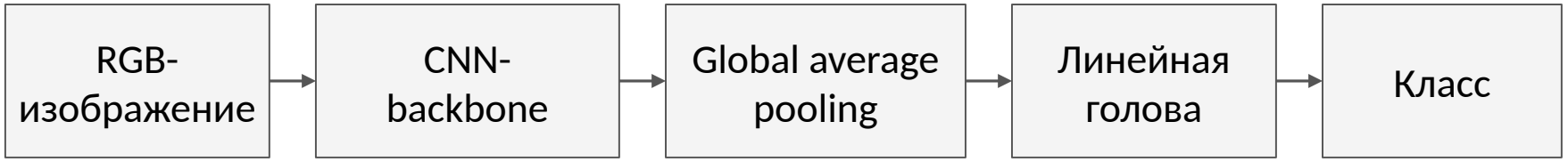
Преобразует признаки в ответ: класс изображения,
рамки и классы объектов или маски сегментации

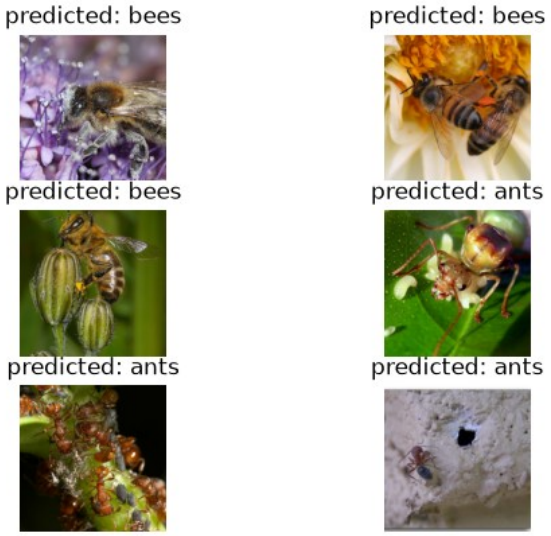
Transfer learning: сохраняем предобученный backbone, заменяем голову и обучаем её под новую задачу.

Источник: https://www.mdpi.com/2504-4990/5/4/83

ResNet и ConvNeXt: классификация

Готовый CNN-backbone строит карты признаков.
Усреднение по пространству и классификатор преобразуют их в ответ.





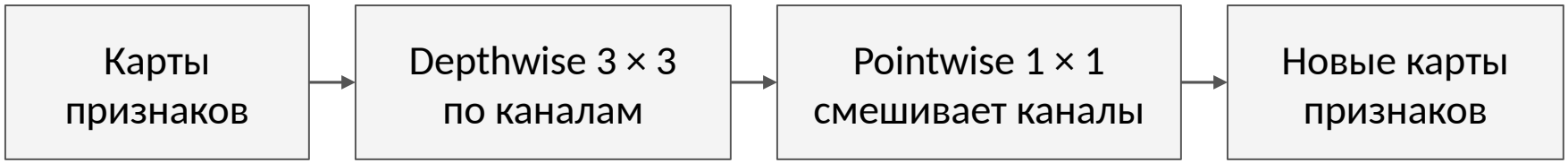
ResNet использует остаточные связи (*residual connections*): они добавляют вход блока к его выходу и облегчают обучение глубоких сетей

ConvNeXt — семейство свёрточных сетей с модернизированными блоками и подходами к обучению

MobileNet и EfficientNet

Компактные модели полезны при ограничениях на память и время обработки изображения. Качество и скорость проверяем на устройстве, где будет работать модель.

Один из экономичных блоков: depthwise separable convolution

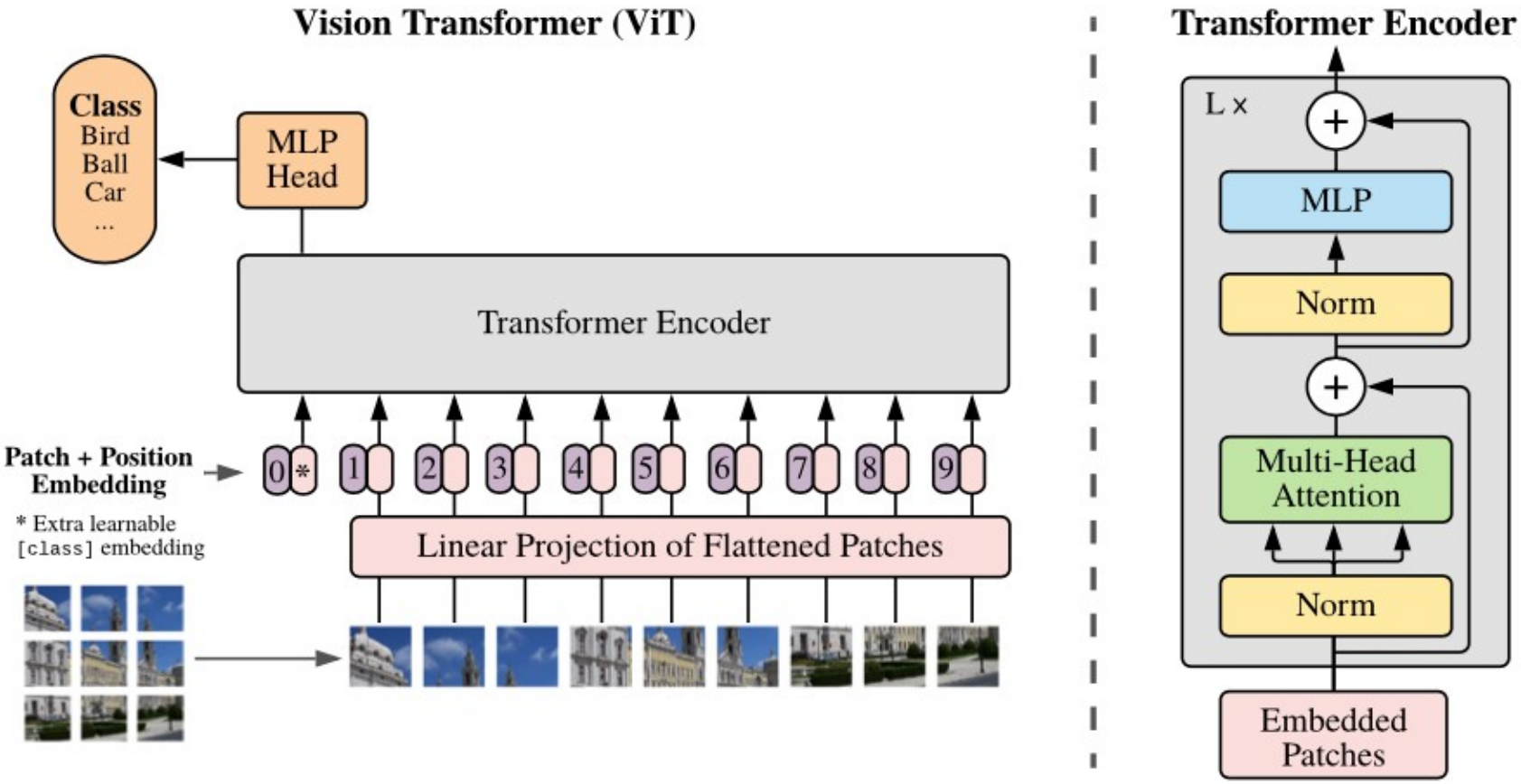


MobileNet использует вычислительно экономичные блоки, в том числе *depthwise separable convolution*: пространственная свёртка обрабатывает каждый канал отдельно, затем свёртка 1×1 объединяет информацию между каналами.

EfficientNet согласованно масштабирует глубину сети, число каналов и разрешение входных изображений. Конкретные блоки зависят от версии семейства.

Небольшое число параметров не гарантирует низкую задержку.

Vision Transformers



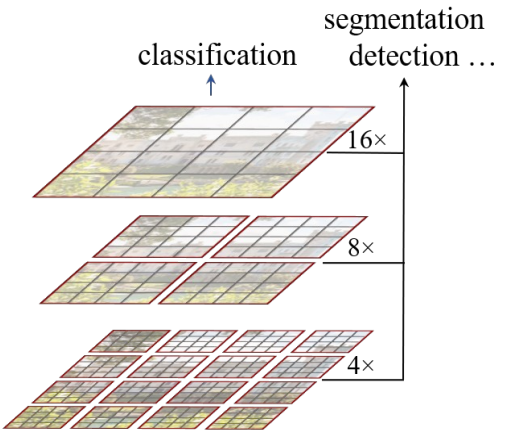
Делит изображение на небольшие фрагменты (*патчи)* и преобразует их в векторы с информацией о положении.
Механизм внимания (*self-attention*) позволяет учитывать связи между разными частями изображения. На основе полученных представлений классификационная голова предсказывает класс изображения

Shifted Window Transformer (SWiN)

Swin вычисляет attention внутри небольших окон изображения, что снижает вычислительные затраты по сравнению с глобальным attention как у ViT

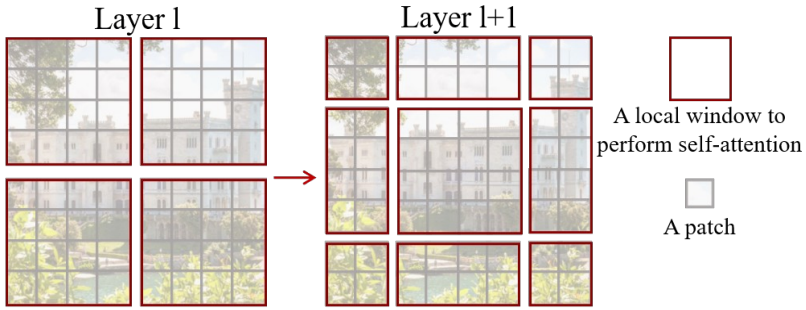
В последовательных блоках границы окон сдвигаются для обмена информацией между соседними окнами.

По мере углубления сети соседние патчи объединяются. Формируется иерархия признаков с разным пространственным разрешением, полезная для распознавания объектов разного размера.



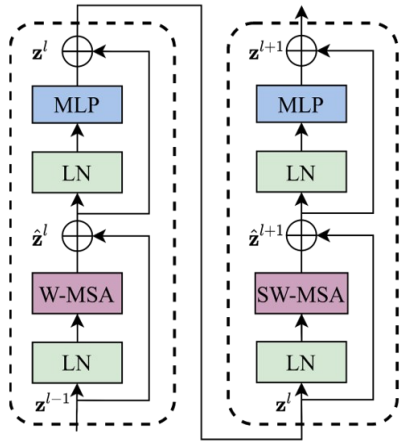
(a) Swin Transformer

$\frac{H}{4} \times \frac{W}{4} \times 48$



(b) Shifted Window

$\frac{H}{4} \times \frac{W}{4} \times C$ $\frac{H}{8} \times \frac{W}{8} \times 2C$



(c) Two Successive Swin Transformer Blocks

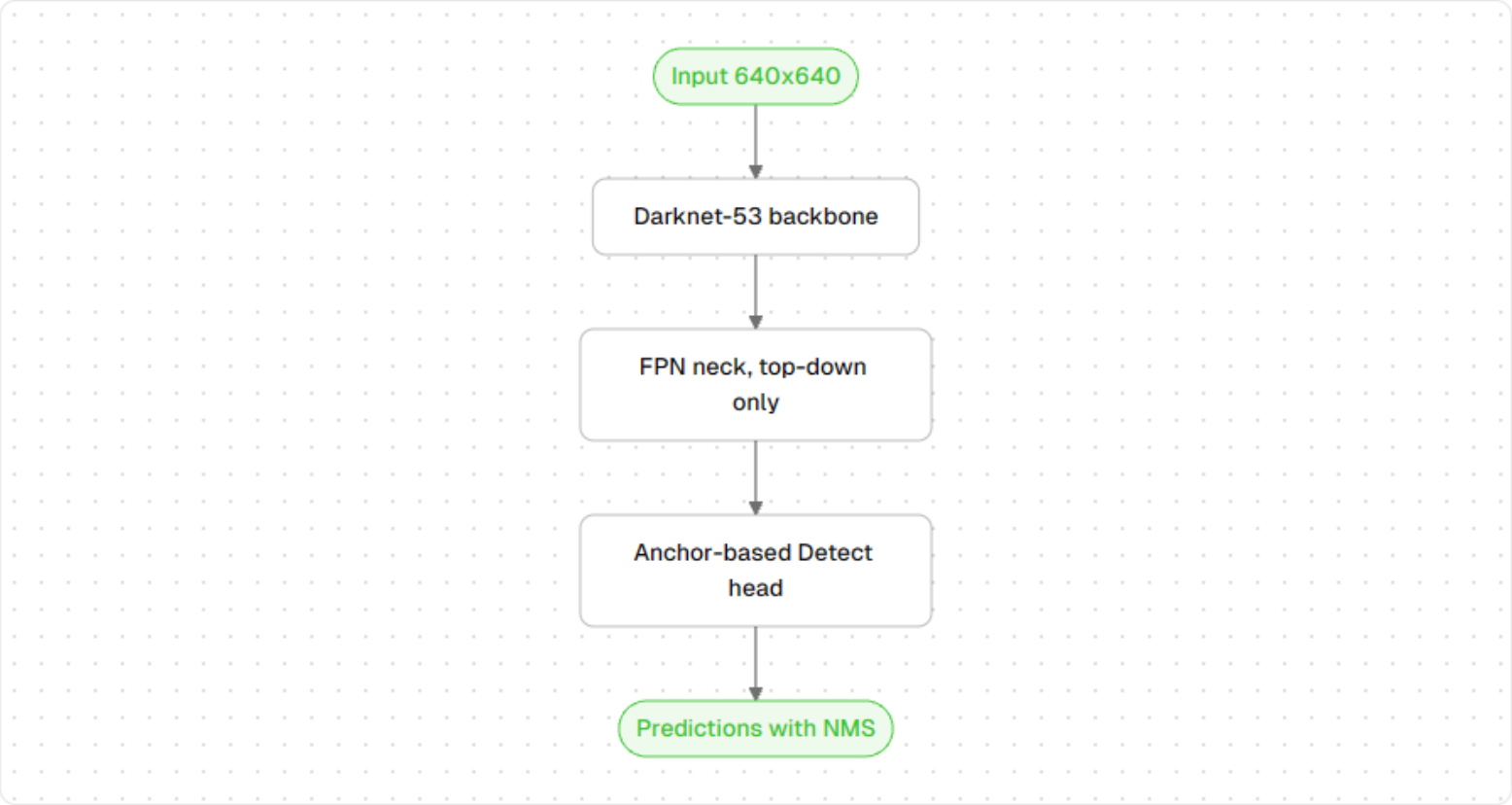
$\frac{H}{32} \times \frac{W}{32} \times 8C$

YOLO (You Only Look Once)

YOLO — семейство моделей для обнаружения объектов.

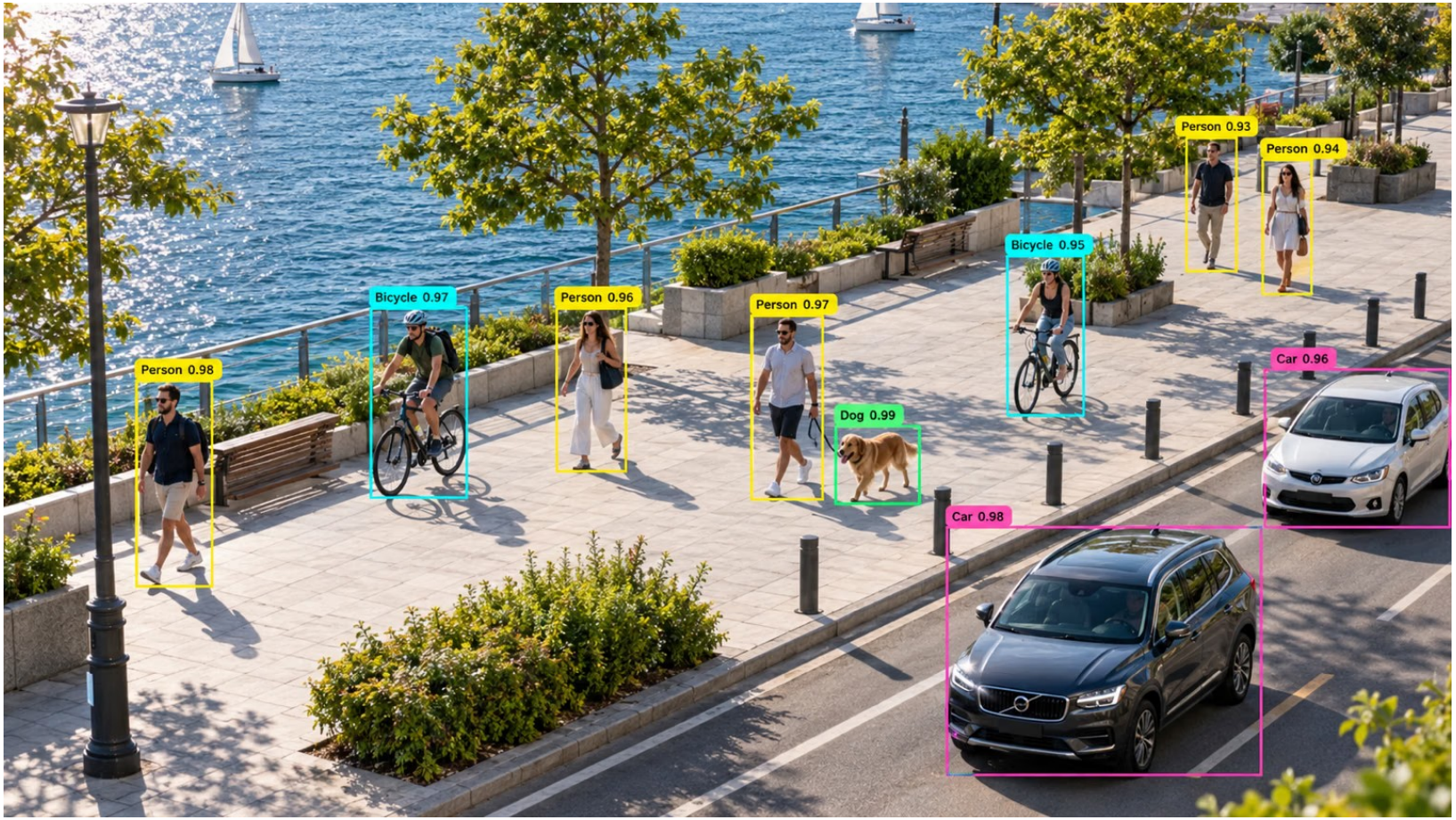
За один проход нейронной сети детектор обрабатывает всё изображение и предсказывает координаты ограничивающих рамок и оценки классов объектов.

YOLOv3 YOLOv5 YOLOv8 YOLO11 YOLO26



Источник: https://docs.ultralytics.com/guides/yolo-architecture

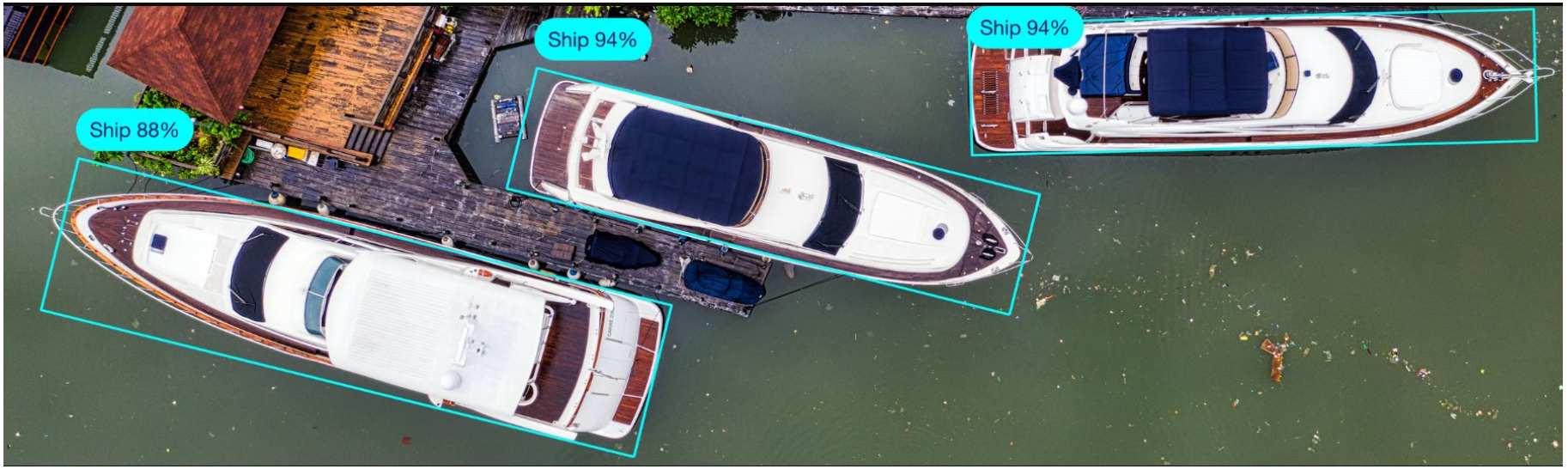
Object Detection



Без дообучения: ищем классы, которые уже знает выбранный checkpoint.
Со своими классами: размечаем рамки и дообучаем готовый детектор.

Источник: https://docs.ultralytics.com/tasks/detect

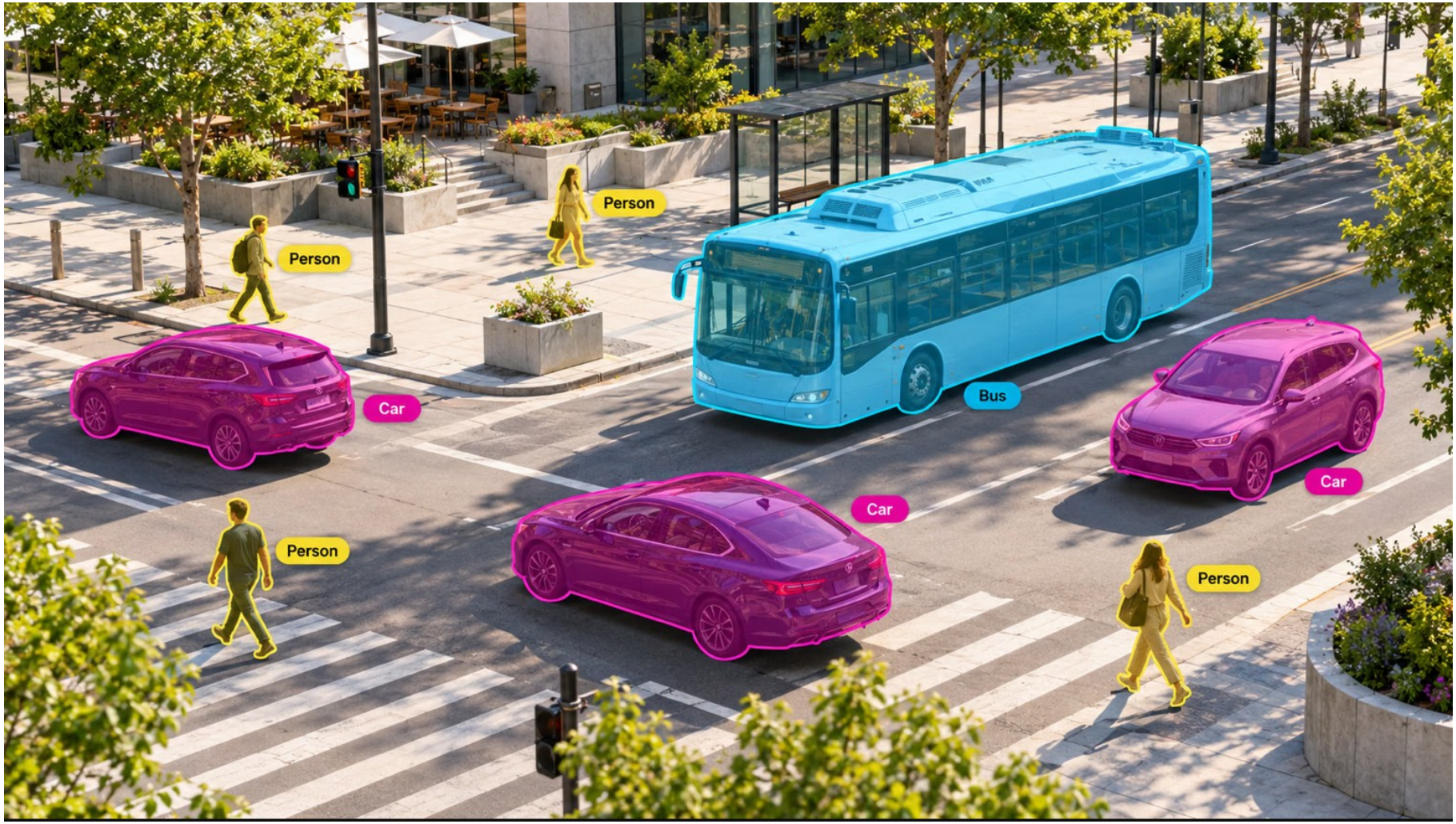
Oriented Bounding Box Detection



Oriented object detection goes a step further than standard object detection by introducing an extra angle to locate objects more accurately in an image.

Источник: https://docs.ultralytics.com

Instance Segmentation



Источник: https://docs.ultralytics.com

Semantic Segmentation



Источник: https://docs.ultralytics.com

Monocular Depth Estimation



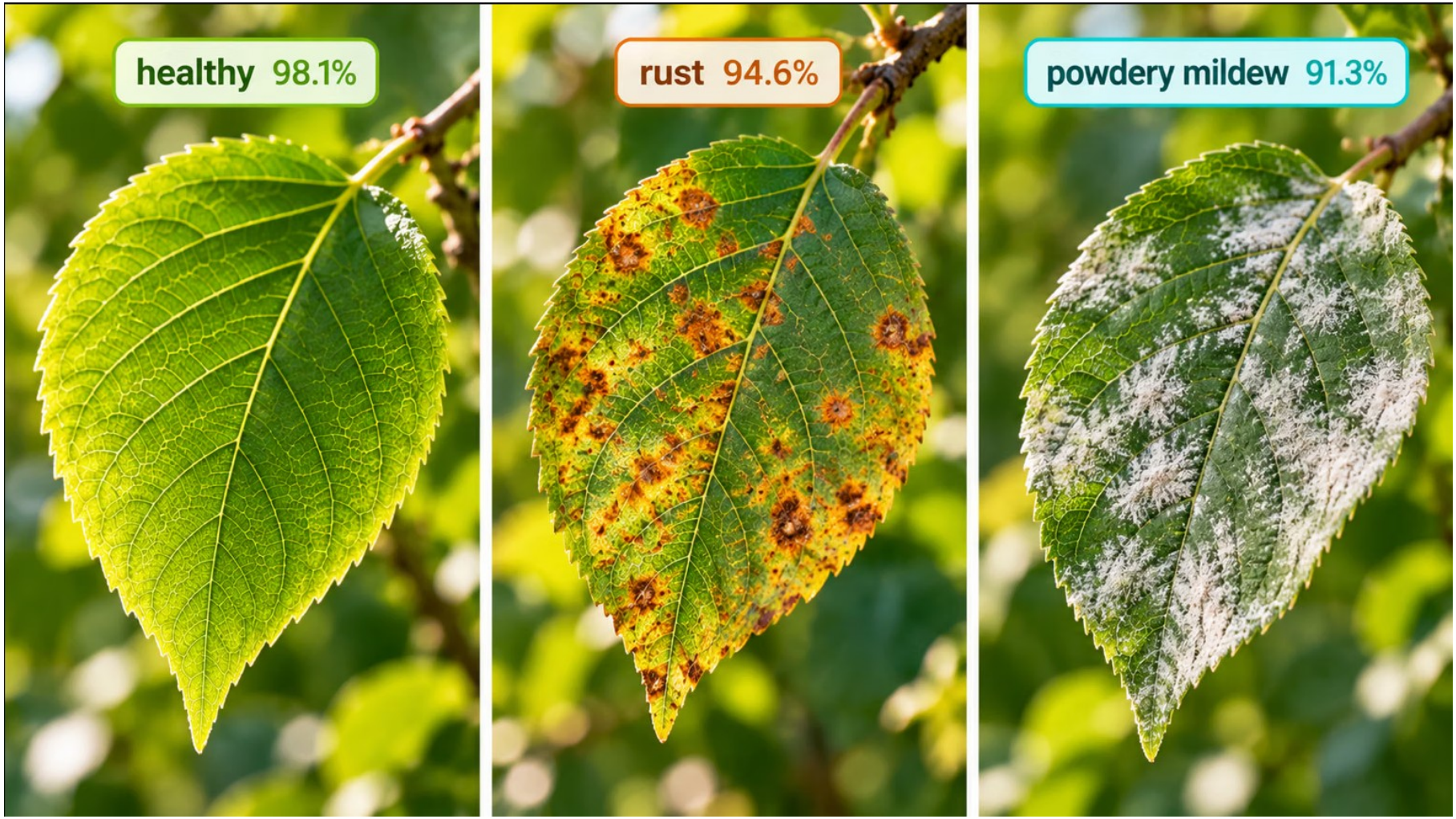


Monocular depth estimation predicts a per-pixel depth map from a single RGB image.
Each output pixel holds a depth value in meters representing the estimated distance from the camera to that surface point.

The output of a depth model is a dense float map of shape (H, W) aligned to the input image. This per-pixel representation makes monocular depth estimation well-suited for 3D scene reconstruction, robot navigation, AR/VR content creation, and any application that requires spatial layout from a single camera.

Источник: https://docs.ultralytics.com

Image Classification



Источник: https://docs.ultralytics.com

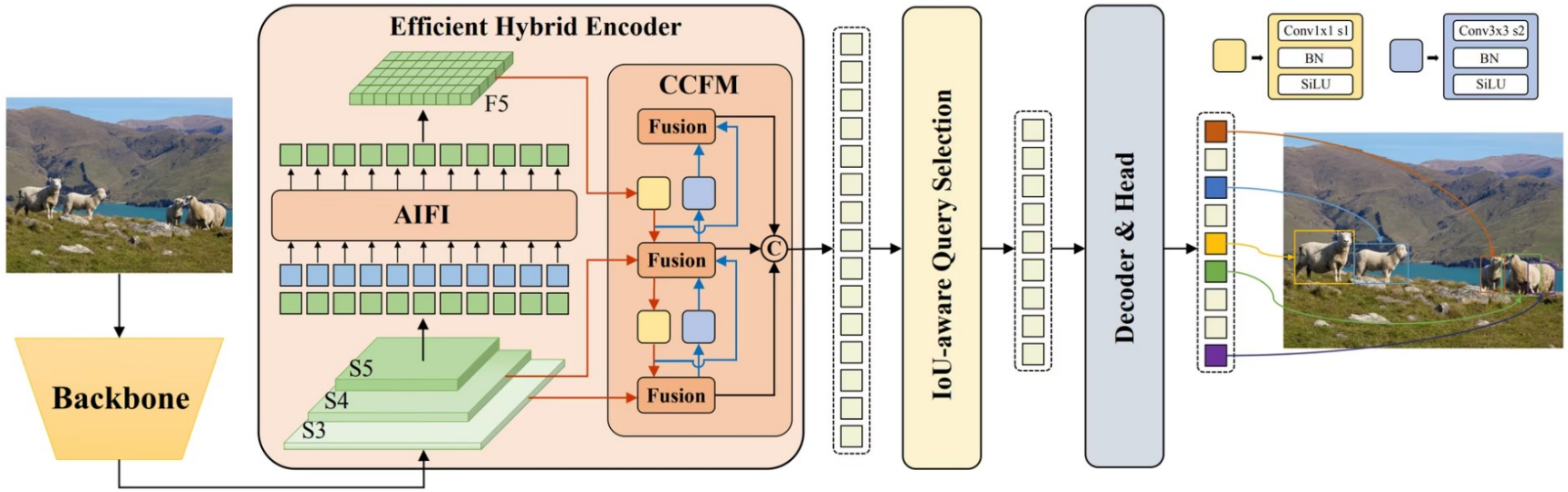
Pose Estimation



Источник: https://docs.ultralytics.com

RT-DETR

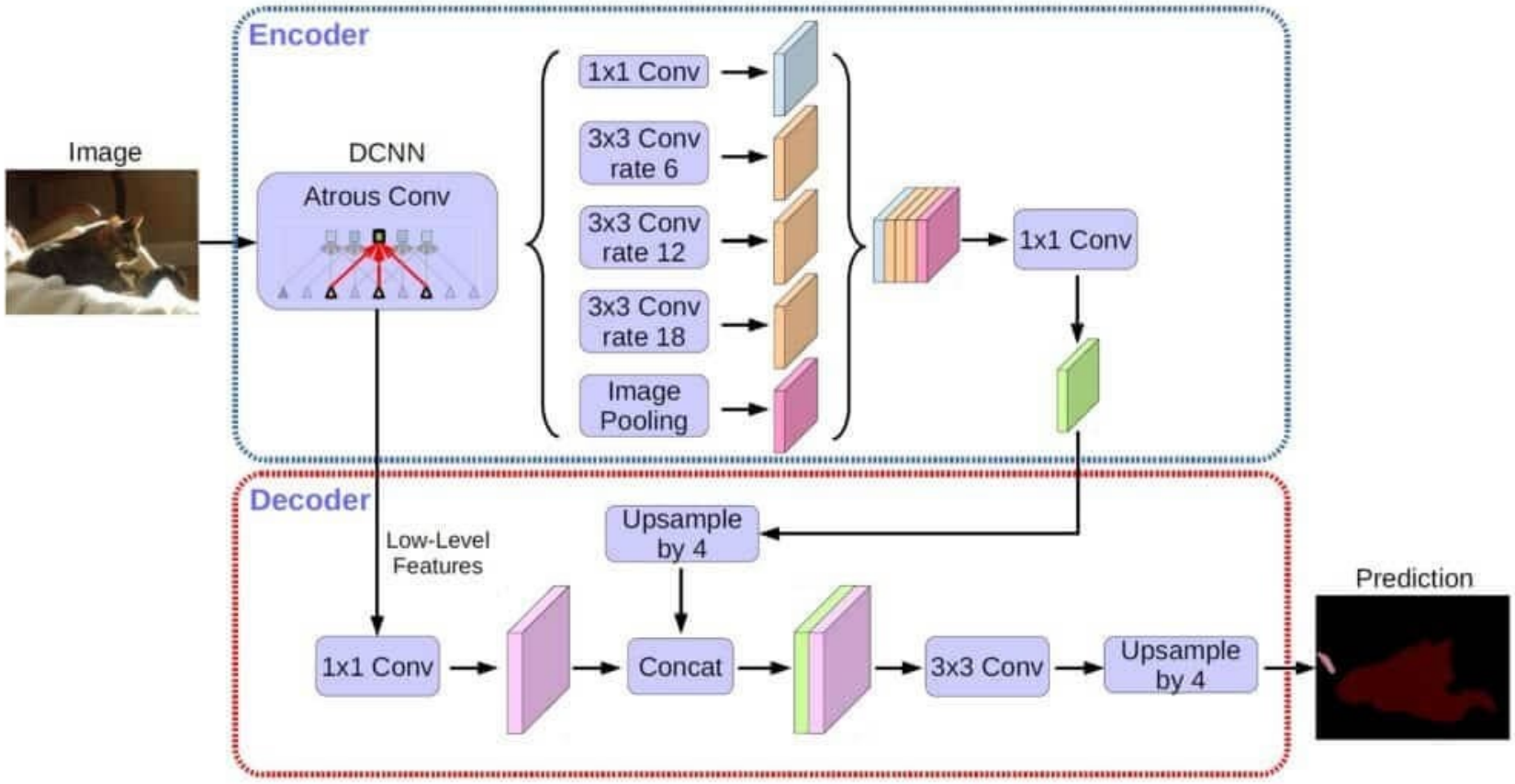
RT-DETR (Real-Time Detection Transformer) - сквозная модель обнаружения объектов, обеспечивающая работу в реальном времени и высокую точность.
Модель основана на DETR — системе обнаружения объектов на трансформерах, которая не использует NMS. Для достижения скорости работы в реальном времени в ней используются свёрточная базовая сеть и эффективный гибридный кодировщик. RT-DETR эффективно обрабатывает признаки разных масштабов, разделяя взаимодействие внутри масштаба и объединение признаков между масштабами.



Источник: https://docs.ultralytics.com/ru/models/rtdetr

DeepLabV3: семантическая сегментация

Модель назначает класс каждому пикселю. Несколько объектов одного класса могут входить в общую маску этого класса.



DeepLabV3: семантическая сегментация

Исходные изображения





Маска класса «собака»



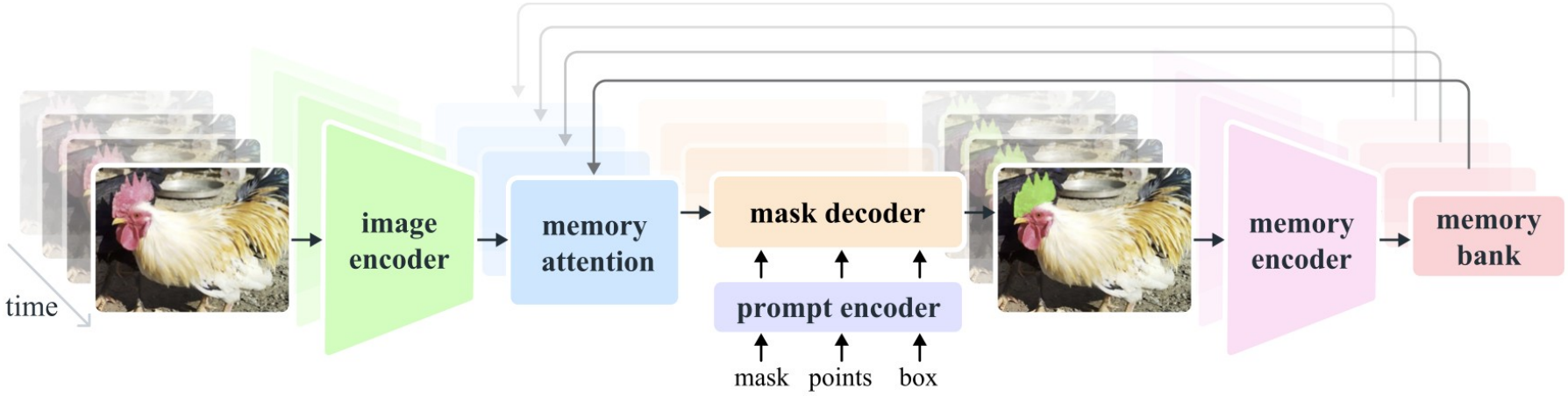


Готовая модель выделяет известный класс. Новый класс требует адаптации.

SAM / SAM 2

SAM (segment anything model) - представила сегментацию изображений по запросу: получив запрос в виде точки, рамки или маски, модель возвращает маску указанного объекта. При этом легковесный декодер работает достаточно быстро для интерактивного использования.

Image encoder извлекает признаки кадра. Prompt encoder кодирует точки, рамку или маску, а mask decoder строит маску объекта. Память помогает использовать информацию из предыдущих кадров. Готовые веса позволяют интерактивно выделять объекты на фото и переносить маску по видео. Человек уточняет объект подсказками.



Уточнение маски точками



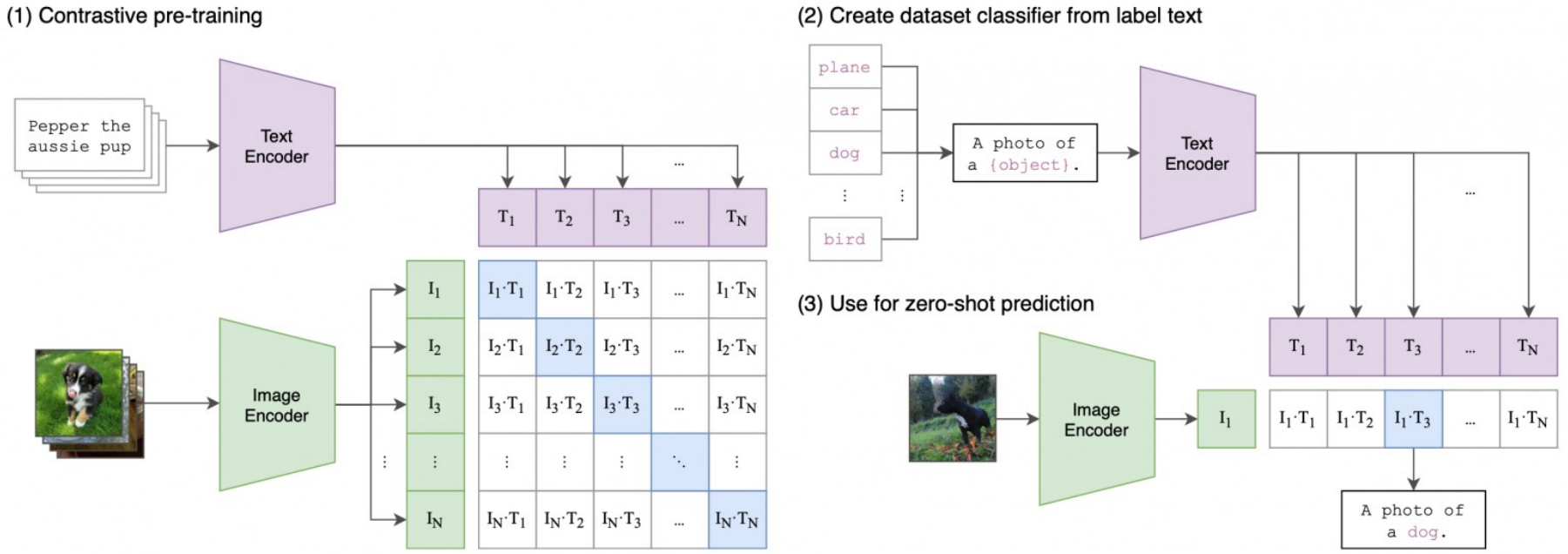
Зелёная точка отмечает нужную область, красная исключает лишнее.
Синяя область показывает полученную маску окна.

Для разметки человек проверяет маску и отдельно назначает класс.

CLIP и SigLIP: сопоставление изображений и текста

CLIP и SigLIP — это мультимодальные нейросетевые модели (Vision-Language Models), которые связывают изображения и текст в общем векторном пространстве.

- Автоматическая разметка (zero-shot) без участия людей.
- Поиск похожих изображений/дубликатов .
- Связь текста и изображений в генеративных задачах

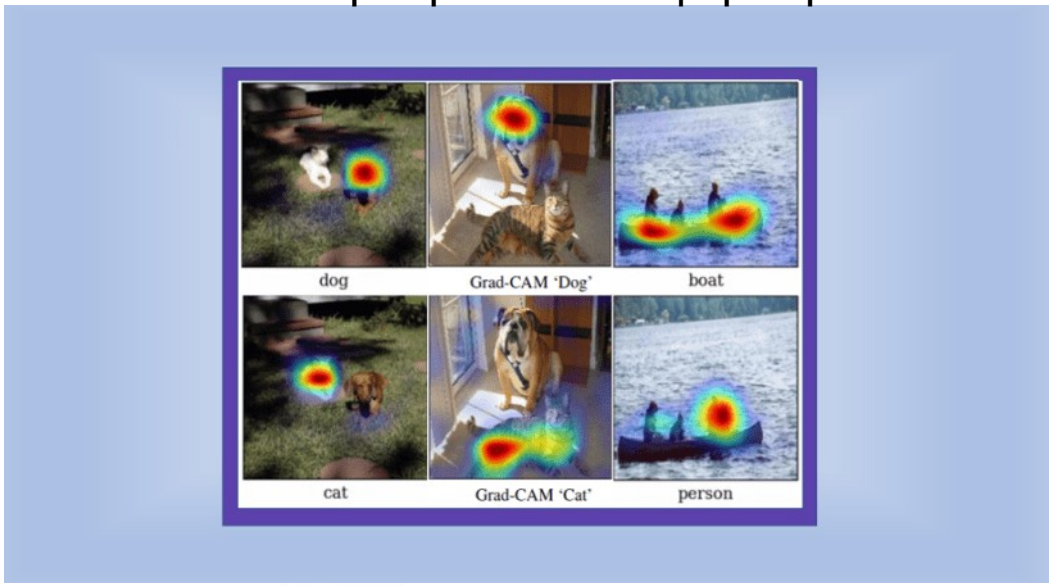


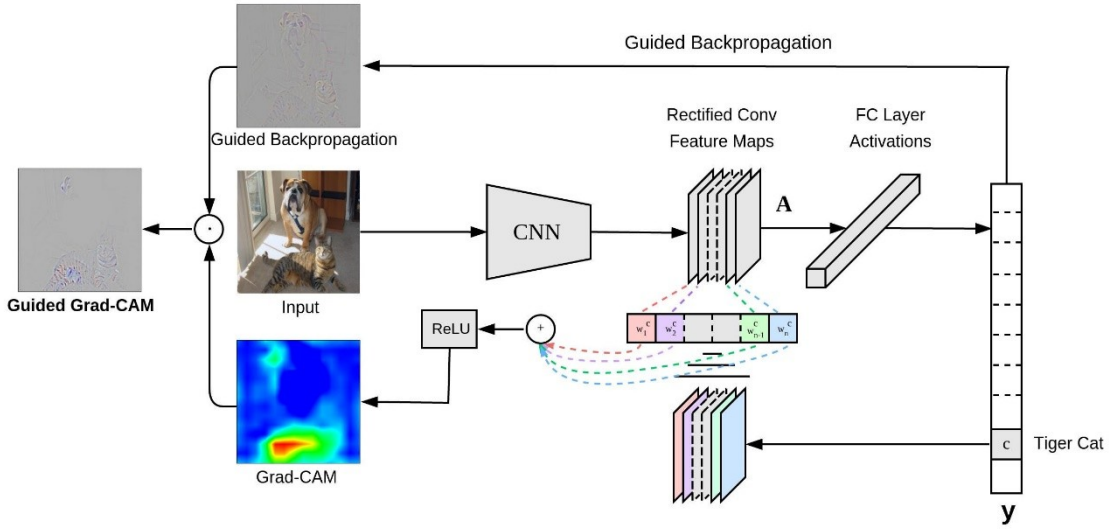
CNN explainer – наглядное средство визуализации CNN



Интерпретация моделей
Gradient-weighted Class Activation Maps (Grad-CAM)

Метод визуализации областей изображения, значимых для предсказания выбранного класса. Использует градиенты оценки класса для определения значимости карт признаков и формирования тепловой карты.

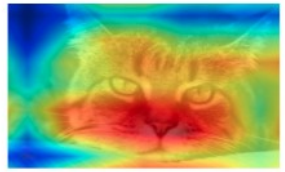




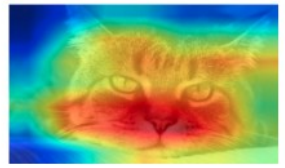
Input image



Class activation map



Grad-CAM



Guided Backpropagation



Guided Grad-CAM

